

Abstract

The aim of this project is to predict the heart diagnosis from ECG signals on the CPSC-2018 dataset. The dataset contains 6877 patients across 9 unique diagnosis codes. We will use machine learning and deep learning models trained on 12-lead ECG signals to classify each patient into one of the diagnostic categories. Model performance will be evaluated using metrics such as accuracy, F1 score, and ROC curve.

Description of the dataset

.mat files

Each patient has a .mat file which stores the raw ECG signal as a matrix. The number of samples and sampling frequency may vary between patients. This file contains the actual electrical activity of the heart across all 12 leads and will be used as input to the machine learning models. The file cannot be opened as a text file and requires a library such as wfdb to read it in Python.

.hea files

Table 1 – describes the basic technical parameters of the overall test, such as the total recording duration, the number of EKG leads used, and the date it was performed.

Table 2 - provides the instruction manual for the computer on how to read the raw numbers from each specific EKG cable and correctly scale them into an electrical voltage graph.

Table 3 - provides the patient's medical metadata, including their age, gender, and diagnosis.

Table 1.

ID	Identification number of this specific test
Number of Leads	Information about number of cables used in the test

Sampling Frequency	The number of times per second a continuous analog signal is measured.
Total Samples	Total number of data points
Data & Time	The exact date and time the patient was hooked up to the ECG machine.

Table 2.

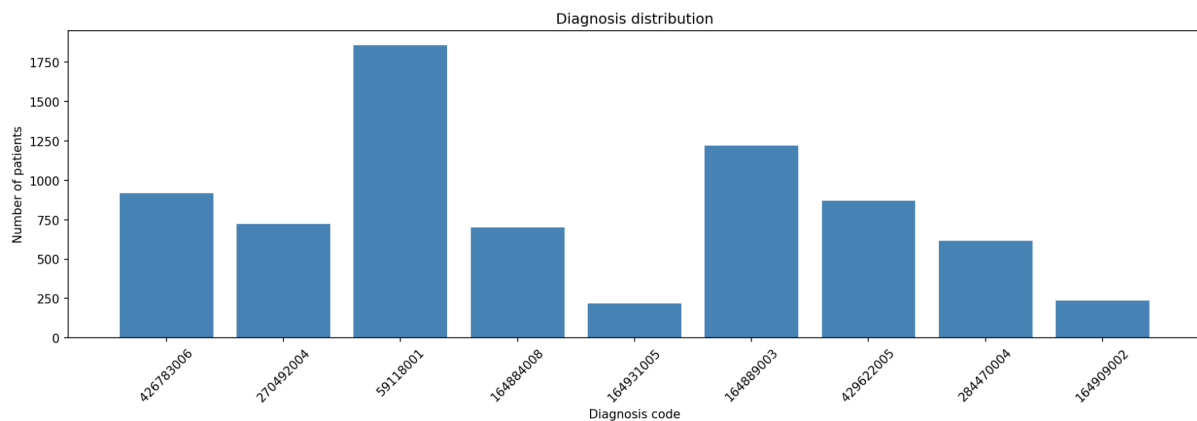
ID	Identification number of this specific test
Storage Format	how bits are laid out in memory.
Scale	The ratio defining how many digital units in the file equal 1 millivolt (1 mV) of real analog heart electricity.
ADC Resolution	The bit-depth of the analog-to-digital converter, indicating signal precision.
Baseline	The digital value that represents exactly zero electrical activity (the flat line).
Initial Value	The value of the very first data sample at the start of the recording.
Checksum	A mathematical value calculated over all samples to verify that the file is not corrupted.
Block Size	A formatting indicator defining whether the data stream is split into specific memory blocks.
Lead Label	The standard clinical name identifying which specific ECG electrode captured this signal.

Table. 3

Age	Age of the patient at the time of the recording.
Gender	The biological sex of the patient.
Diagnosis Code (Dx)	A standardized clinical terminology code (usually SNOMED CT) identifying the medical condition.
Medications (Rx)	Documented pharmaceutical treatments or drugs currently prescribed to the patient.
Medical History (Hx)	Past medical background, underlying conditions, or relevant patient history.

Symptoms (Sx)	Presenting clinical complaints or physiological symptoms reported by the patient.
---------------	---

Diagnosis Distribution



Diagnosis (Dx)	patients (num)	
59118001	: 1857	The dataset contains 6877 patients across 9 unique diagnosis codes. The most frequent diagnosis is 59118001. The least common diagnosis is 164931005 with 220 cases. The dataset is notably imbalanced; the most common class has over 8x more samples than the rarest one. This will need to be addressed during model training, for example by using oversampling techniques or class weights in the loss function.
164889003	: 1221	
426783006	: 918	
429622005	: 869	
270492004	: 722	
164884008	: 700	
284470004	: 616	
164909002	: 236	
164931005	: 220	

426783006 - (Normal Sinus Rhythm) is the only diagnosis code in this dataset that represents a healthy heart. All other 8 codes indicate various heart conditions such as atrial fibrillation, bundle branch blocks, or ventricular hypertrophy.

59118001 - (Bundle branch block (disorder))

164889003 - (Electrocardiographic atrial fibrillation (finding))

429622005 - (ST Depression (observable entity))

270492004 - (Partial atrioventricular block (disorder))

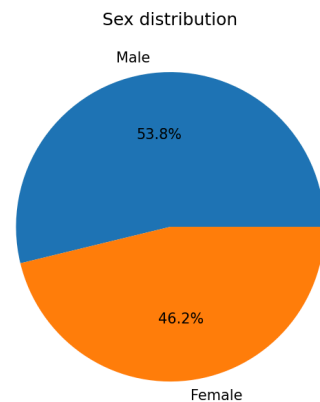
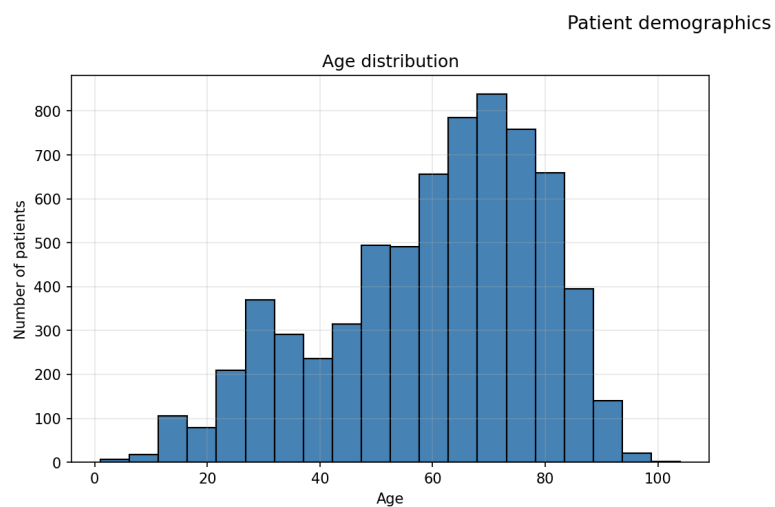
164884008 - (Electrocardiogram: ventricular ectopics (finding))

284470004 - (Premature atrial contraction (disorder))

164909002 - (Electrocardiographic left bundle branch block (finding))

164931005 - (ST elevation (observable entity))

Patient Demographics



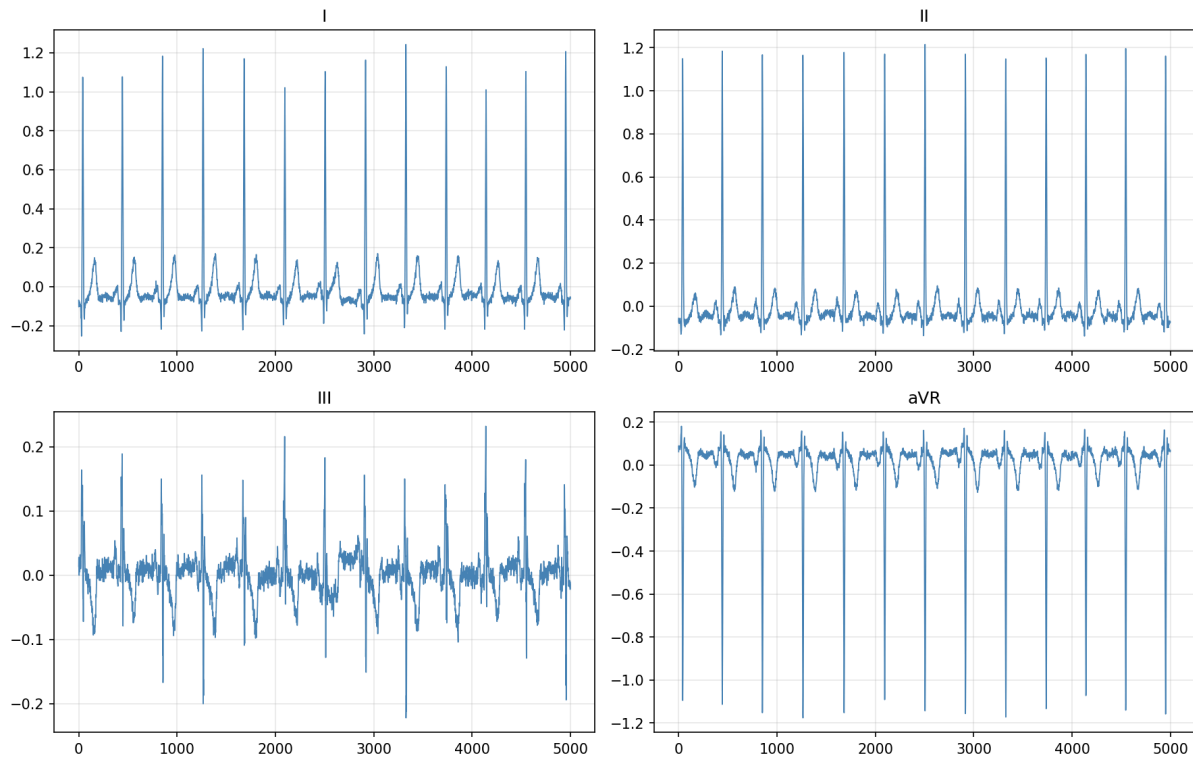
Age groups:

0-20: 189
21-40: 1024
41-60: 1730
61-80: 3004
81+: 921

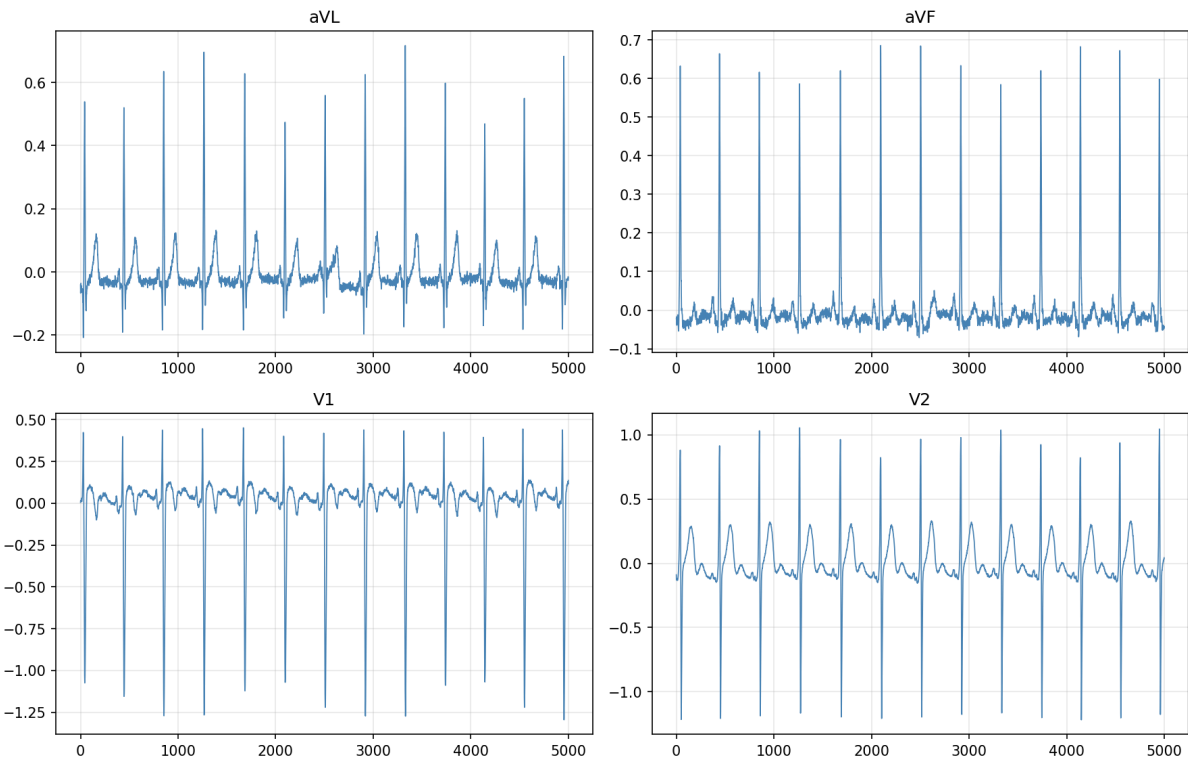
Average age: 60.2

The dataset consists of 6877 patients with a fairly balanced sex distribution — 53.8% male and 46.2% female. The average age is 60,2 years, with the majority of patients falling in the 61-80 age group (3004 patients). The youngest group (0-20) is the least represented with only 189 patients, which reflects the fact that heart conditions are significantly more common in older populations.

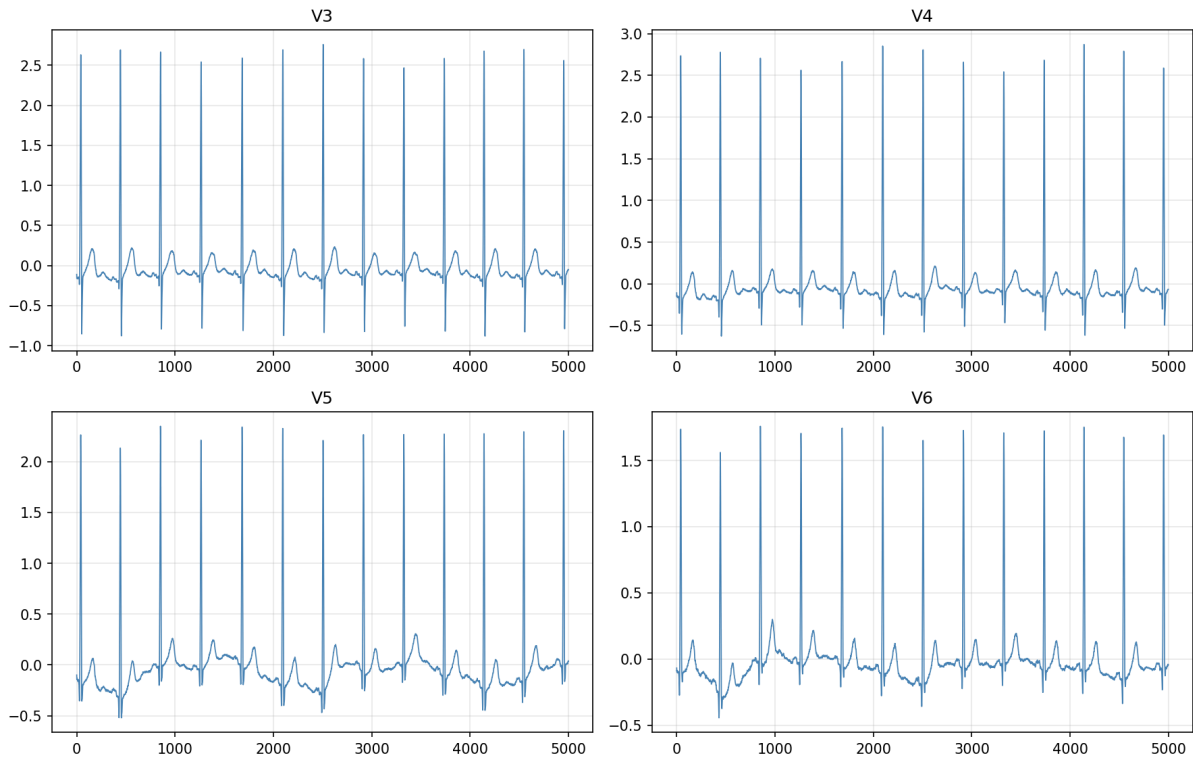
ECG: A2112 | Age: 55 | Sex: Male | Dx: 426783006 | Part 1/3



ECG: A2112 | Age: 55 | Sex: Male | Dx: 426783006 | Part 2/3

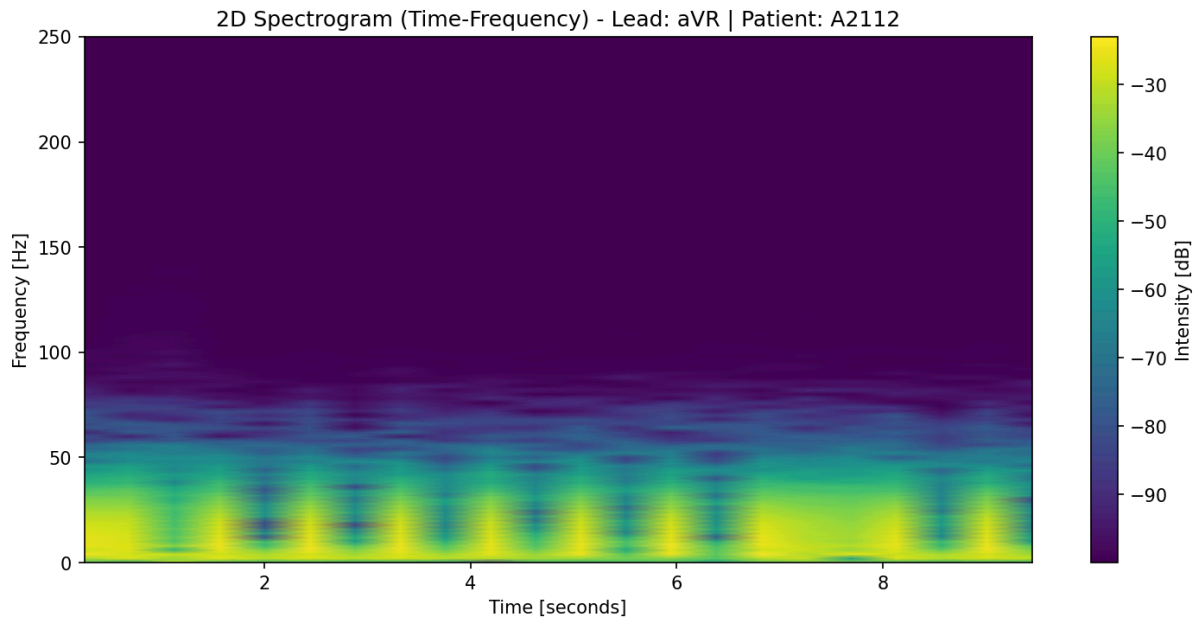


ECG: A2112 | Age: 55 | Sex: Male | Dx: 426783006 | Part 3/3



ECG Sample Visualization — Patient A2112

The figures show ECG signals of patient A2112, a 55-year-old male with diagnosis 426783006 (Normal Sinus Rhythm). The recording is split into 3 parts, each showing 4 out of 12 leads. Each lead records the electrical activity of the heart from a different angle. The signal spans 10 seconds with 5000 data points sampled at 500 Hz. The sharp peaks visible across all leads represent heartbeats, which appear regular and evenly spaced, which is expected for a patient with normal sinus rhythm.



2D Spectrogram (Time-Frequency) — Lead: aVR | Patient: A2112

The primary purpose of converting the 1D ECG signal into a 2D spectrogram is to extract both temporal (time) and spectral (frequency) features simultaneously. While 1D signals only show voltage changes over time, the 2D spectrogram exposes hidden frequency patterns—such as the rapid energy bursts of a heartbeat or the continuous low-frequency vibrations of an arrhythmia. This algebraic transformation turns a complex time-series problem into a computer vision task. It allows deep learning models (like 2D-CNNs) to classify heart conditions by recognizing distinct geometric shapes and color distributions instead of analyzing raw numerical rows.

Plan to solve the problem

Dataset Splitting Strategy

The dataset will be split into 80% training and 20% test set. The test set will be locked away and only used at the very end to evaluate the final model performance.

For model training and tuning we will use Stratified K-Fold Cross Validation.

Stratified version is chosen because the dataset is imbalanced — the most common class has over 8x more samples than the rarest one. Stratification ensures that each fold maintains the same ratio of diagnoses as the original dataset.

To further address the class imbalance we will apply SMOTE (Synthetic Minority Oversampling Technique) on the training set only. SMOTE creates new artificial samples for minority classes by interpolating between existing samples and their nearest neighbors, rather than simply duplicating them. This prevents the model from being biased towards the most common diagnosis.

Multi-Lead Data Integration and Proposed Classifiers

Since the CPSC-2018 dataset utilizes standard 12-lead ECG recordings, the models must process all 12 signals simultaneously to guarantee multi-class diagnostic accuracy. Before classification, each of the 12 denoised time-series leads is independently transformed into a 2D Mel-Spectrogram of size 128x128.

Algorithms

1. Baseline Classifier: Support Vector Machine (SVM)

Data Integration: All 12 spectrogram matrices are flattened individually into 1D arrays and concatenated horizontally into a single, comprehensive 1D feature vector per patient. This results in a final vector containing 196,608 unique features (12 leads multiplied by 128 multiplied by 128 pixels)

2. Custom 2D Convolutional Neural Network (2D-CNN)

Data Integration: The 12 individual spectrograms are stacked together along the channel dimension, creating a multi-channel tensor of shape (12, 128, 128) per patient. This acts

exactly like a 12-channel image, where each channel represents a unique spatial view of the heart's electrical activity.

3. Deep Transfer Learning: ResNet-18

Data Integration: Similar to the Custom CNN, data is fed as a stacked (12, 128, 128) multi-channel tensor. The input layer of the standard network is modified to accept 12 channels instead of the standard 3 RGB channels.

Main Tools, Frameworks, and Libraries

WFDB (Waveform Database): Used to programmatically read and parse the clinical raw data matrices from the .mat files and extract relevant patient metadata from the corresponding .hea header files.

Scikit-learn: Used for splitting the dataset, implementing Stratified K-Fold cross-validation, and performing the SMOTE oversampling technique on the training set. It will also provide the Support Vector Machine (SVM) implementation and calculate core classification metrics

PyTorch / Keras: Used as the primary deep learning frameworks to construct, compile, and train the Custom 2D-CNN architecture as well as to load the pre-trained ResNet-18 model for transfer learning.

NumPy & Pandas: Employed for efficient high-dimensional array handling, matrix transformations, and restructuring structural patient metadata.

Matplotlib : Utilized to generate demographic charts, signal graphs, 2D spectrogram heatmaps, and final evaluation plots like the ROC curve.

SciPy: Applied during the preprocessing and time-frequency analysis stages. Specifically, `scipy.signal.butter` and `scipy.signal.filtfilt` will handle baseline wander removal, while `scipy.signal.spectrogram` will perform the short-time Fourier transformations.

Evaluation Methods and Performance Comparison

To strictly assess the performance of the three implemented classifiers and ensure a reliable multi-class diagnostic system, models will be validated using Stratified K-Fold Cross-Validation on the 80% training partition. The remaining 20% test set will be kept completely locked and utilized exclusively at the very end to compute the final, unbiased generalization performance.

Given that the CPSC-2018 dataset is highly imbalanced—with the dominant diagnosis appearing over eight times more frequently than the rarest one—relying solely on simple classification accuracy would be highly misleading. Therefore, a comprehensive set of classification metrics will be computed using a macro-averaged approach across all 9 diagnostic categories:

Accuracy: To measure the overall percentage of correctly predicted diagnostic categories across the entire evaluation set.

Specificity: To measure the model's ability to correctly identify true negative cases, ensuring that healthy patients or patients with distinct conditions are not misdiagnosed with the wrong defect.

[1] F. F. Liu *et al.*, "An open access database for evaluating the algorithms of ECG rhythm and morphology abnormal detection," *Journal of Medical Imaging and Health Informatics*, vol. 8, no. 7, pp. 1368–1373, 2018.

[2] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000, doi: 10.1161/01.CIR.101.23.e215.

Implementation

In order to streamline the process and reduce time spent on processing, we decided to split the process into three distinct stages:

- Preprocessing - Precomputing spectrograms for each input record and saving the output for later use
- Training and Selection - Training a select model
- Evaluation - Evaluation of model performance and chart generation

Preprocessing

The goal of the preprocessing stage was to process the raw training data into a unified format we can easily train our models on.

During development, the preprocessing stage evolved into two variants: a memory-optimized full dataset baseline and an experimental version aimed at increasing the number of samples underrepresented classes.

We chose the approach of generating a spectrogram for each ECG lead, and converted the notes containing the condition diagnosis into an array of binary labels using a `MultilabelBinarizer` from `scikit-learn`. It should be noted that there can be more than one label per input record, making this a multilabel-multioutput classification task, and we treated it as such afterwards.

Before creating spectrograms, the ECG signals were filtered to reduce noise. Then each lead was converted into a spectrogram.

In order to improve accuracy our preprocessing pipeline had several hyperparameters we could tweak to generate distinct datasets for evaluation.

nperseg	Amount of segments used during the spectrogram generation stage
length_cap	Minimum length of viable samples in seconds. Shorter samples are filtered out from the dataset
split_extra_samples	Whether to generate additional samples through segmenting samples longer than length_cap into segments. This procedure is only done for labels that were uncommon and had poor detection performance, which go as follows: • "164931005" • "284470004"(V2) / "270492004"(V1) • "426783006" • "429622005"

After initial generation the datasets were saved for later usage in training as pickle files, with the spectrogram being saved as a numpy tensor of 12 stacked 2d arrays of equal length, each array representing one lead.

For the memory optimization full dataset baseline, the precomputed file stored only the processed spectrogram arrays and labels, without the full WFDB record objects. This made the saved data smaller and allowed repeated training runs without recomputing spectrograms.

Initially we intended to address the class imbalance via SMOTE, however in practice we realized that it does not function well in case of multilabel, multioutput problems, necessitating either a bespoke generation strategy out of scope for this project or omitting multioutput cases from the generation pool, as a result we decided to postpone synthetic data generation for later, in case we were unsatisfied with the model performance

Because of these limitations, the first complete pipeline focused on consistent spectrogram preprocessing, stratified data splitting, and model comparison, while later preprocessing variants explored segmentation-based oversampling for selected rare labels.

Datasets

During our process we created several precomputed datasets which will be mentioned by name during the following stages.

Precomputed-256

This dataset replicates the initial flawed conditions for our experiment

Name	precomputed-256
nperseg	256
length_cap	6
split_extra_samples	False

Precomputed-nperseg-64

First dataset that resulted in successful model training

Name	precomputed-nperseg-64
nperseg	64
length_cap	6
split_extra_samples	False

Precomputed

Iteration upon the precomputed-nperseg-64 dataset, used as baseline.

Name	precomputed
nperseg	32
length_cap	6
split_extra_samples	False

Precomputed - Length 10, Oversampling

Experimental dataset applying both optimizations suggested during the troubleshooting process. Length 10 was used as the vast majority (>97%) of records satisfy the condition, filtering out only very short samples of length 6s, which was the minimum value as defined by the original paper

Name	precomputedsplit-genlength-10
nperseg	32
length_cap	10
split_extra_samples	True (V1)

Precomputed - Oversampling

Dataset with the extra sample generation optimization, in the initial version label 270492004 was oversampled instead of 284470004 due to a mistake, despite having good detection rates.

Name	precomputed-split-gen-length-6-old
nperseg	32
length_cap	6
split_extra_samples	True (V1)

Precomputed - Oversampling (V2)

Dataset with the extra sample generation optimization, using proper labels as per the intention behind the extra class generation.

However in all cases the V2 variant of oversampling performed significantly worse, hence V1 oversampling was used for most datasets

Name	precomputed-split-gen-length-6
nperseg	32
length_cap	6
split_extra_samples	True (V2)

Precomputed - Length 10

Dataset which only includes the longer training data without extra class generation via oversampling records with lower frequency classes

Name	precomputed-length-10
nperseg	32
length_cap	10
split_extra_samples	False

Training and Selection

After preprocessing, we started to compare several model architectures. Since the task was multilabel classification, each model produced one output for each diagnosis class.

In the full dataset baseline pipeline, the data was first split into a training-validation part and a separate holdout test set using multilabel stratified split. This helped preserve similar

diagnosis proportions across the splits, which was important because the dataset was highly imbalanced.

The first model used as a baseline was Support Vector Machine. Since SVM models cannot directly process spectrogram images, the 12 spectrograms were flattened into a single one-dimensional feature vector for each patient. During training, the SVM frequently produced warnings, which suggested that the model struggled to fully optimize on such high-dimensional flattened spectrogram data.

The second model was a 2D Convolutional Neural Network. In this case, the spectrograms were treated as image inputs, allowing the network to learn local time-frequency patterns from the ECG signals.

The third model was based on deeper CNN architectures, including a ResNet model and later CNN-LSTM variants. These models were tested whether deeper modeling could improve performance compared with the simpler CNN baseline.

For model selection the full dataset baseline, SVM, CNN, and ResNet-like CNN were compared using stratified cross-validation on the training-validation data. The best model type was selected based on validation performance and was then trained again before final evaluation on the separate holdout test set.

During later experiments, additional architectures and preprocessing variants were tested, including CNN variants, CNN-LSTM models, different segment lengths, and oversampling-based datasets.

The full dataset baseline also allowed quick control over experiment size using a sample-size parameter. Smaller runs were used to verify that the pipeline worked, while the final baseline evaluation was performed on all patient records.

Evaluation

Because the project was a multilabel classification task, model evaluation could not rely only on standard accuracy. A prediction was considered more complex than a single class decision, because each patient could have several diagnosis labels at the same time. We evaluated the models using several multilabel metrics and visualizations.

For the full dataset baseline, the final model was evaluated on a separate holdout test set that was not used during model selection. This made the final score more reliable because the model was tested on patient records that were not used to choose the best architecture.

The main metrics included Subset Accuracy, Micro F1, Macro F1, Samples F1, Jaccard score, and Hamming score. Subset Accuracy is the strictest metric, because it only counts a prediction as correct if the entire set of predicted labels matches the true set of labels for a patient. This makes it difficult to achieve high values in multilabel problems.

Micro F1 measures performance globally across all labels, while Macro F1 calculates the score separately for each label and then averages the results. Macro F1 is especially useful in this project because it gives more visibility to rare diagnosis classes. Samples F1 evaluates performance per patient, and Jaccard score measures how much the predicted label set overlaps with the true label set. Hamming score shows how often individual label decisions were correct.

Training and Results

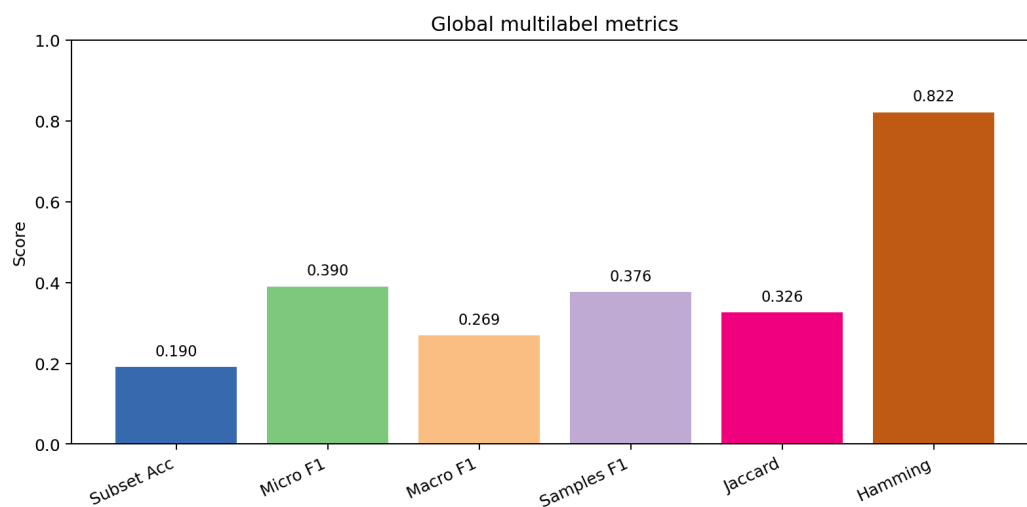
Initial Experimentation

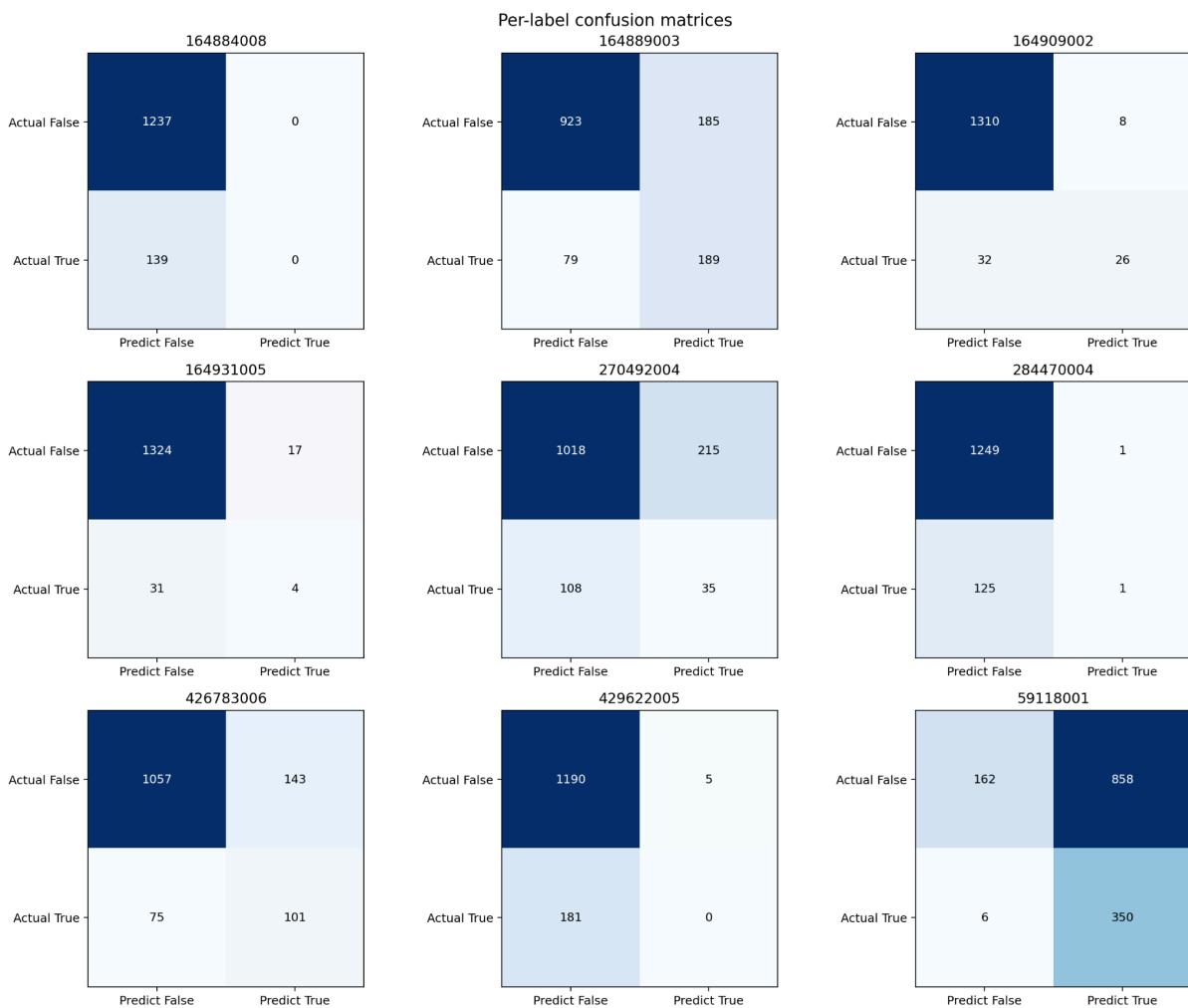
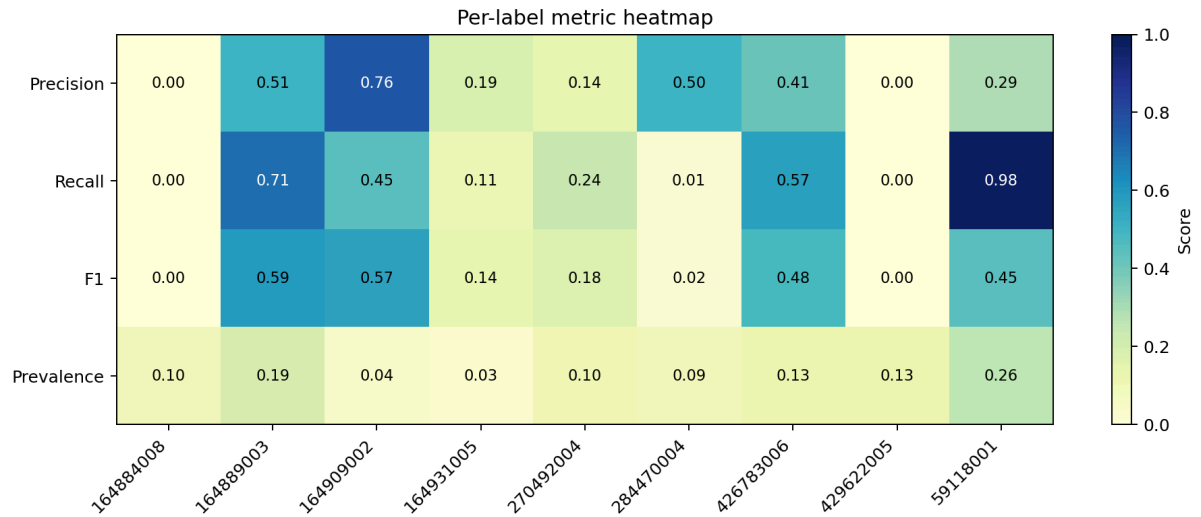
Once the models and general framework were implemented initial training attempts were conducted. However to our dismay the models did not learn, the training was carried out, and the metrics were promising, however in practice the model returned 0 for uncommon cases, and the learning was incorrect for more well represented classes.

The issue was present in both the scikit SVM model as well as the keras-based 2DCNN and ResNet networks.

An output of a sample run using the (eventually best performing) 2D CNN model on the given dataset resulted in the following metrics:

Experiment Name	cnn_nperseg256
dataset	precomputed-256
Model	2d-CNN
Variant	default
iterations	10





From these results it is obvious that the models are not learning properly.
We investigated several strategies to mitigate this problem:

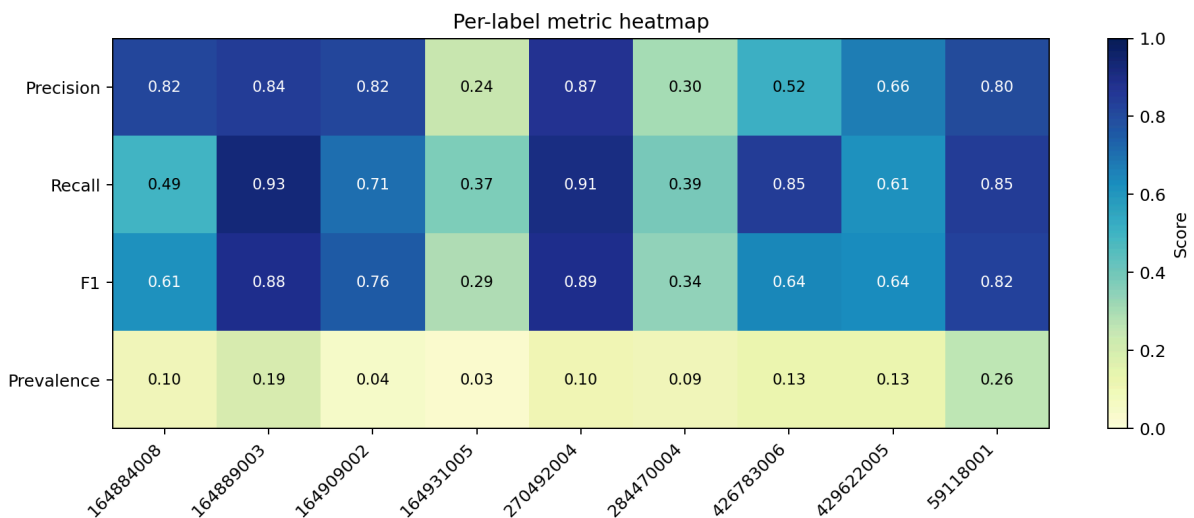
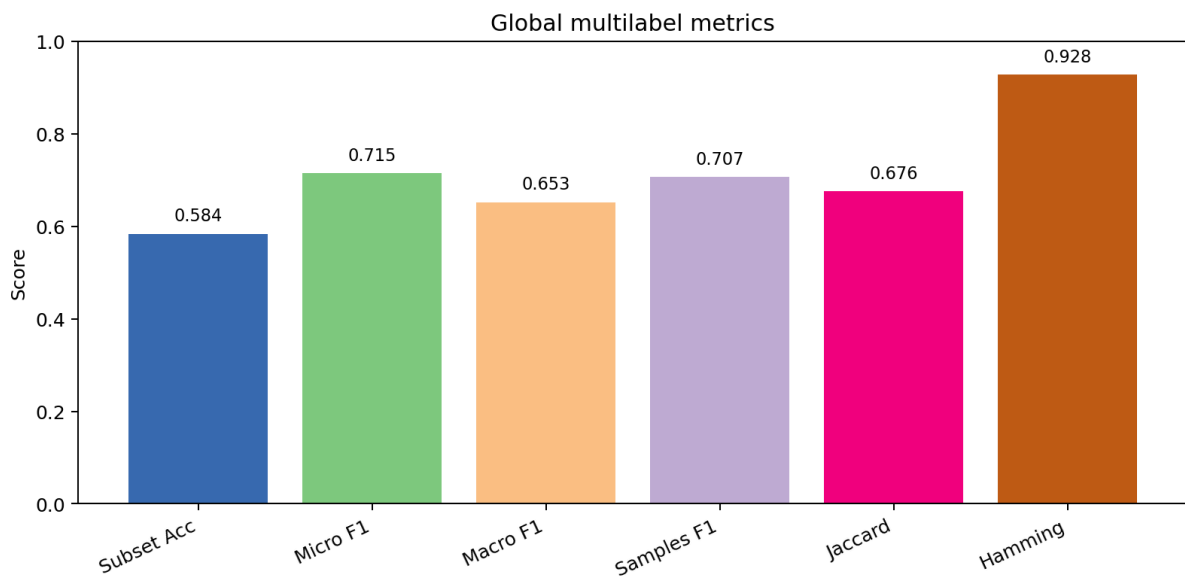
- Changing the keras loss function
- Explicitly using HE weight initialization
- Changing model architecture by adjusting layer count and dimensions

- Changing activation functions for output
- Implementing variable thresholds

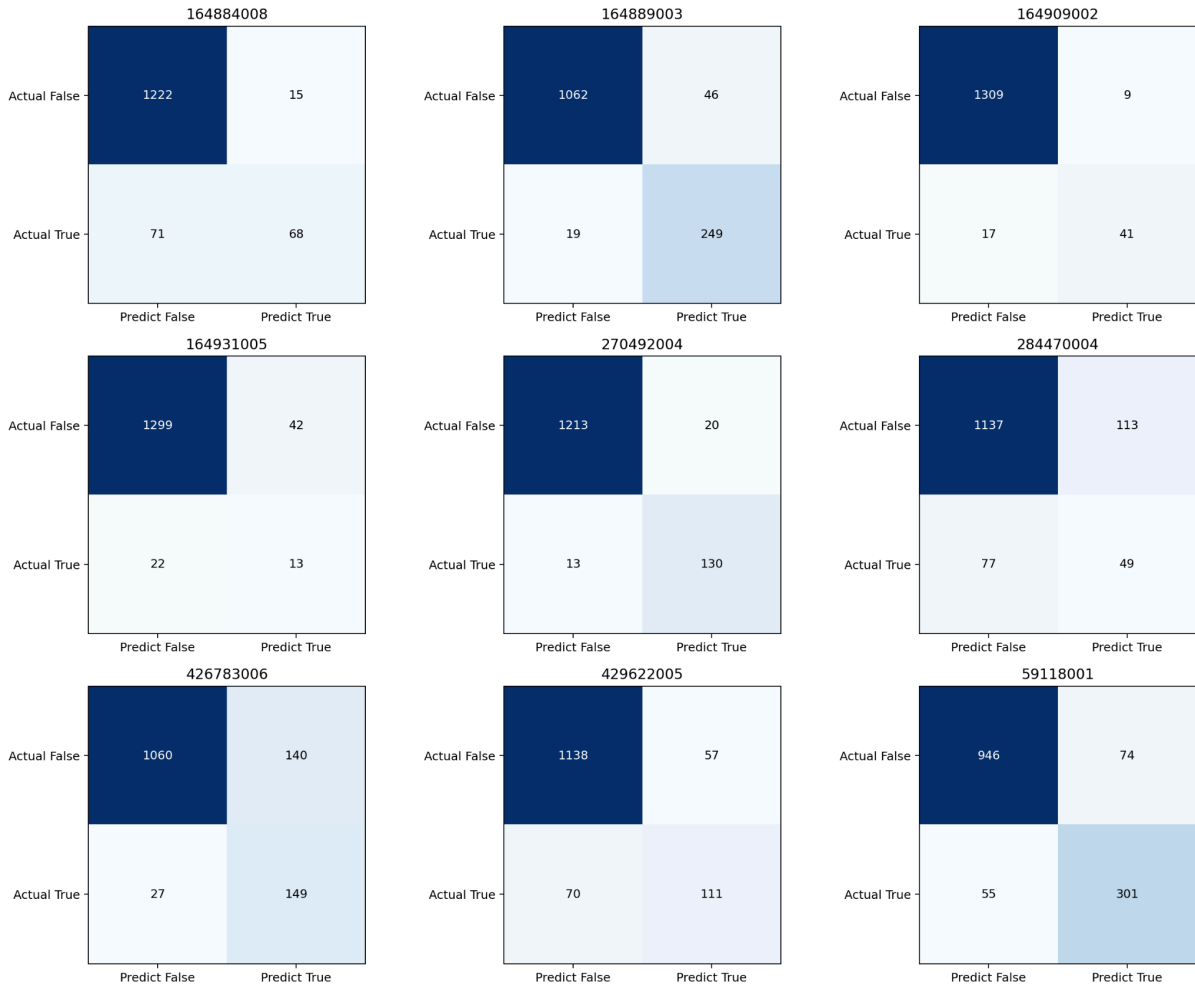
After excluding all other explanations we narrowed the problem down to the precomputed training input data. While nperseg 128 or 256 seem to be standard for spectral classification tasks, in practice we did not yield good results as long as we stuck to them.

Drastic improvements were observed once the training data was regenerated with nperseg set to 64, improving further with nperseg 32

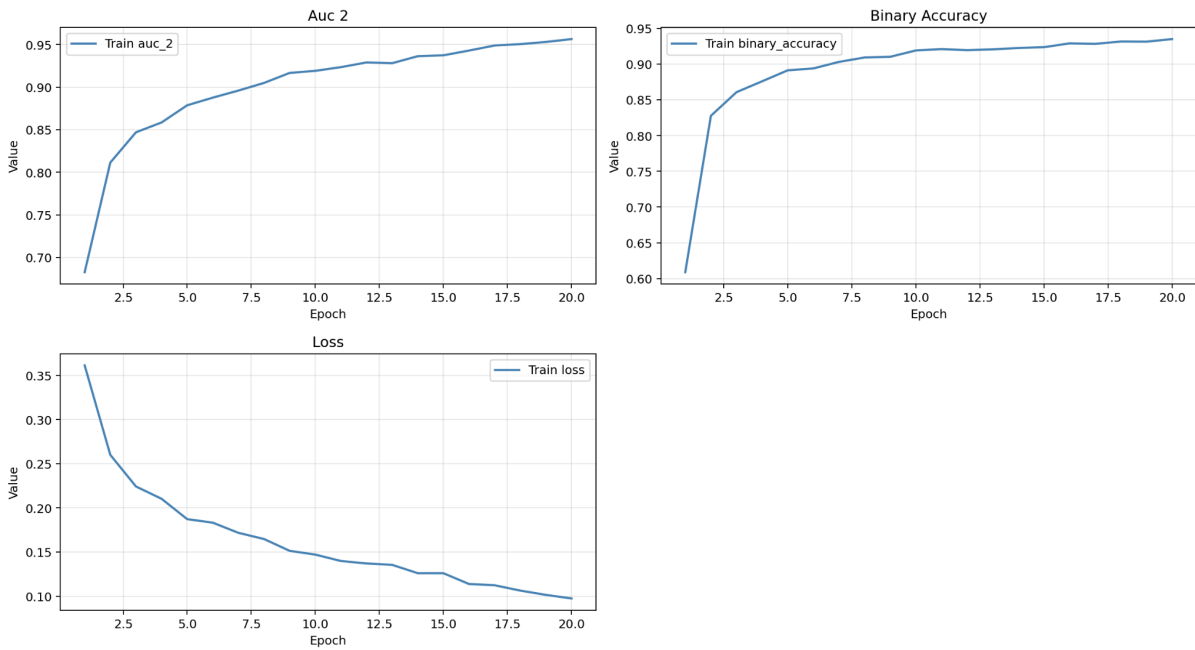
Experiment Name	cnn_nperseg_32
dataset	precomputed
Model	2d-CNN
Variant	default
iterations	20



Per-label confusion matrices



Training history — cnn-default



In this case we can see generally decent performance, across the board, however several cases had poor detection rates, which was taken note of and used for generating extra dataset entries by splitting longer records.

The entries that were chosen for this include:

- "164931005"
- "284470004"(V2) / "270492004"(V1)
- "426783006"
- "429622005"

From this point nperseg 32 was our baseline setting for all future tests

Baseline Pipeline and Full Dataset Evaluation

As a first complete experiment, we implemented a baseline pipeline that processed the full dataset, compared 3 model types, and evaluated the final selected model on a holdout test set. The goal of this stage was to create a solid reference point for later experiments.

The compared models were a Support Vector Machine, 2D CNN, and a ResNet-like CNN. The SVN used flattened spectrograms, while CNN-based models used the spectrograms' image-like image inputs. Model selection was performed using stratified cross-validation, and the best model was later evaluated on a separate test set.

Using all 6877 patient records, the final model achieved a Micro F1 score of 0.491, Macro F1 0.523, Samples F1 0.579, Jaccard 0.475, Hamming 0.778, Subset Accuracy 0.255.

The results show that the model achieved high recall for most diagnosis classes, meaning it detected many true positive cases. However, precision was low for some labels, which means the model also produced many false positive predictions. Therefore, this baseline was sensitive but not selective enough.

This experiment provided a working end-to-end reference point for later improvements, especially experiments on oversampling and different spectrogram parameters.

2D-CNN Iteration and improvement

In our tests we determined that the custom 2d CNN architecture had the best performance, therefore it became our baseline for further experiments to improve model accuracy.

Our models were not ready at the halfway mark for the project, due to the training issues and as a result we received several suggestions for possible improvements.

Despite the issue being specifically located within the training data, these pointers were good suggestions for improvements and as a result we applied them to the best performing model.

We investigated the following optimizations:

- LSTM (Long Short Term Memory) architecture
- Class imbalance resolution (Oversampling)
- Increasing training data length

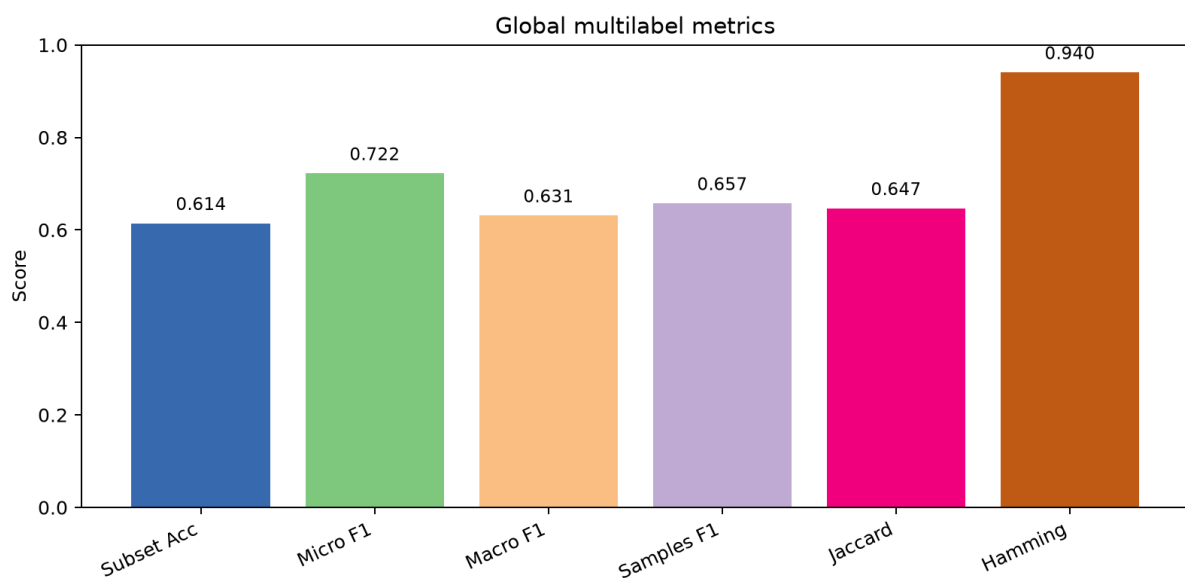
Oversampling (V1/V2)

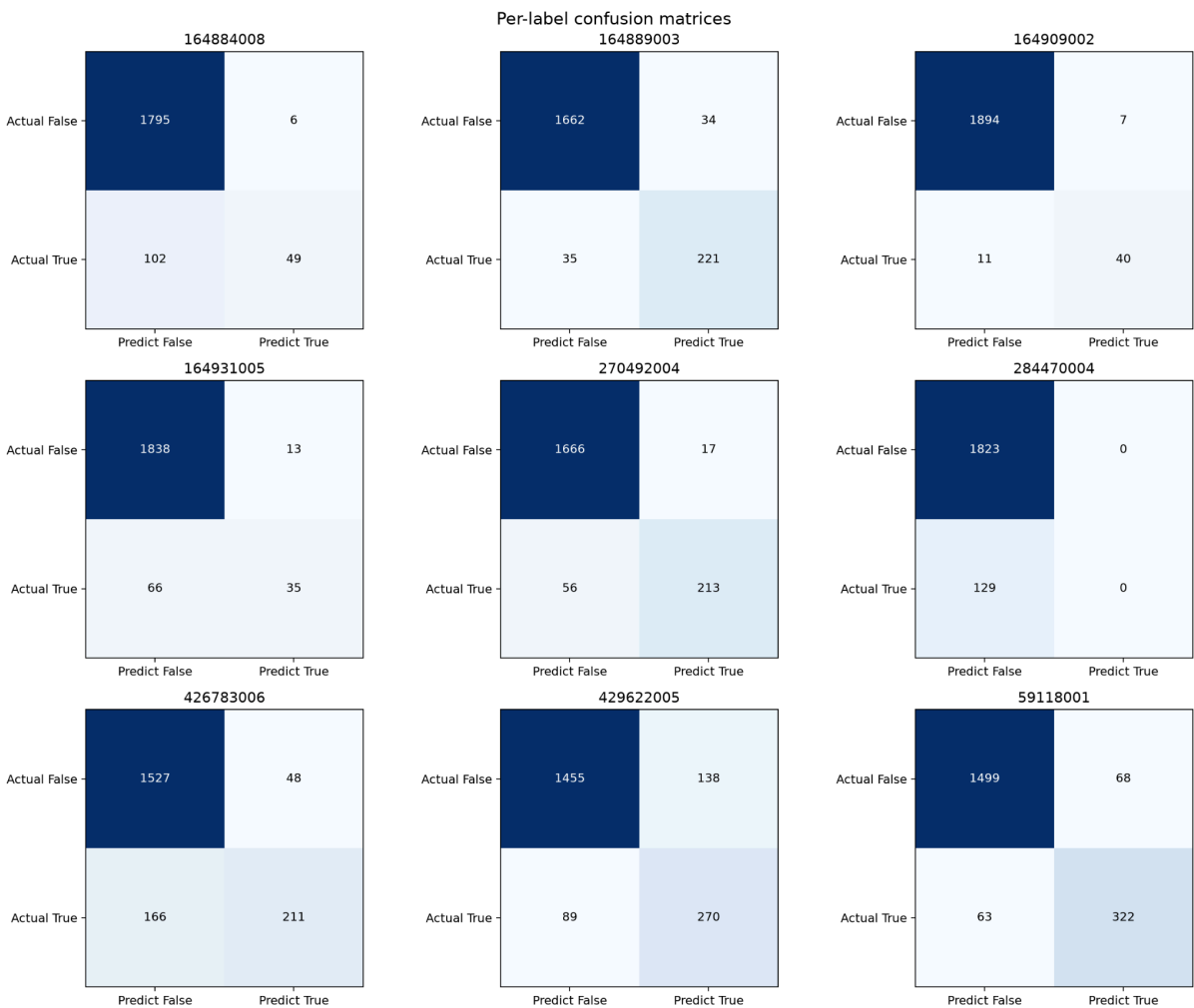
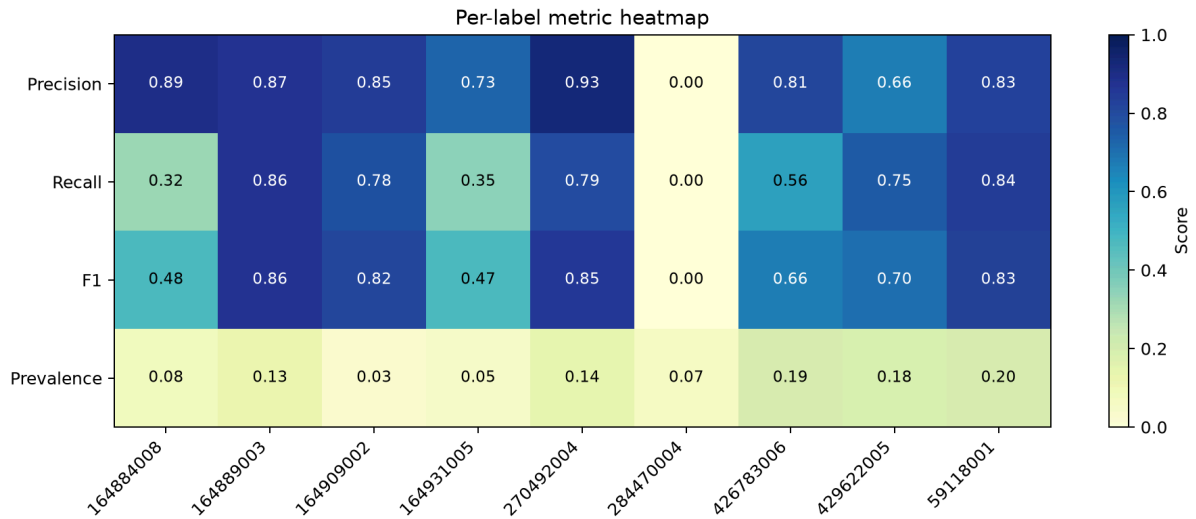
Given the poor performance for certain labels as exposed by the confusion matrices and per-label heatmap, we first attempted to remedy those issues by oversampling these cases in our training data.

As previously mentioned this was achieved by splitting records that were longer than the minimum length into chunks of even length equal to the minimum length.

These chunks were afterwards inserted into the dataset as additional training entries.

Experiment Name	cnn-l6-oversample
dataset	precomputed-split-gen-length-6-old
Model	2d-CNN
Variant	default
iterations	20

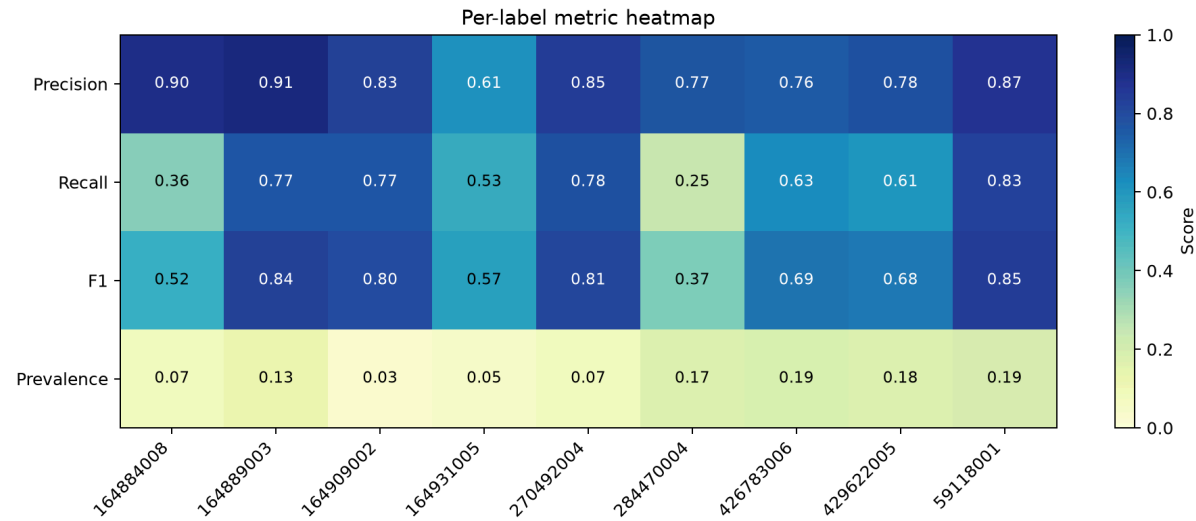
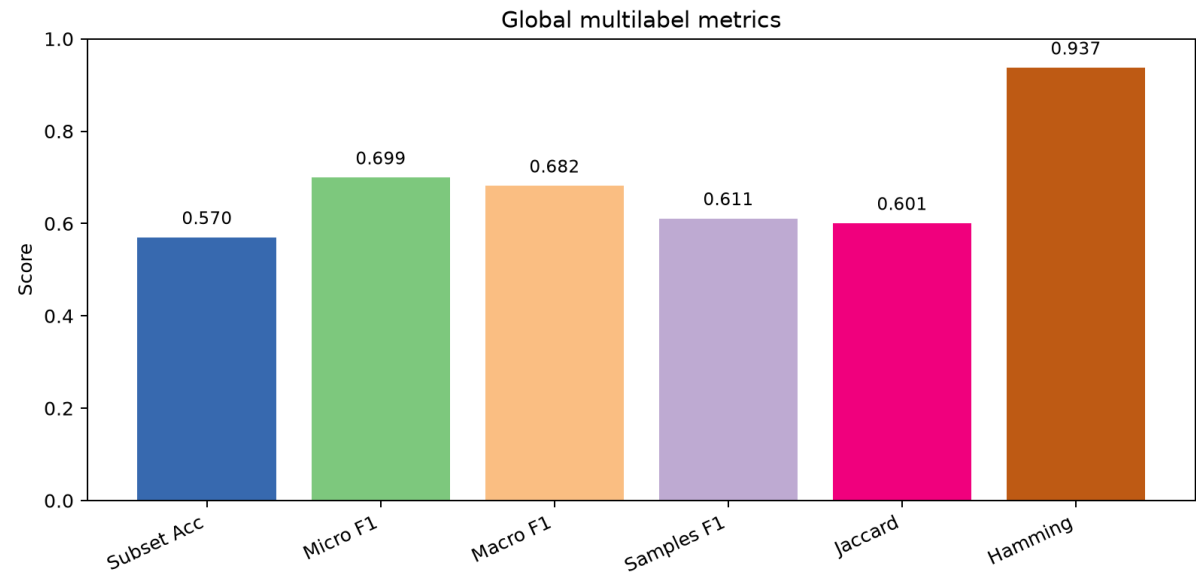


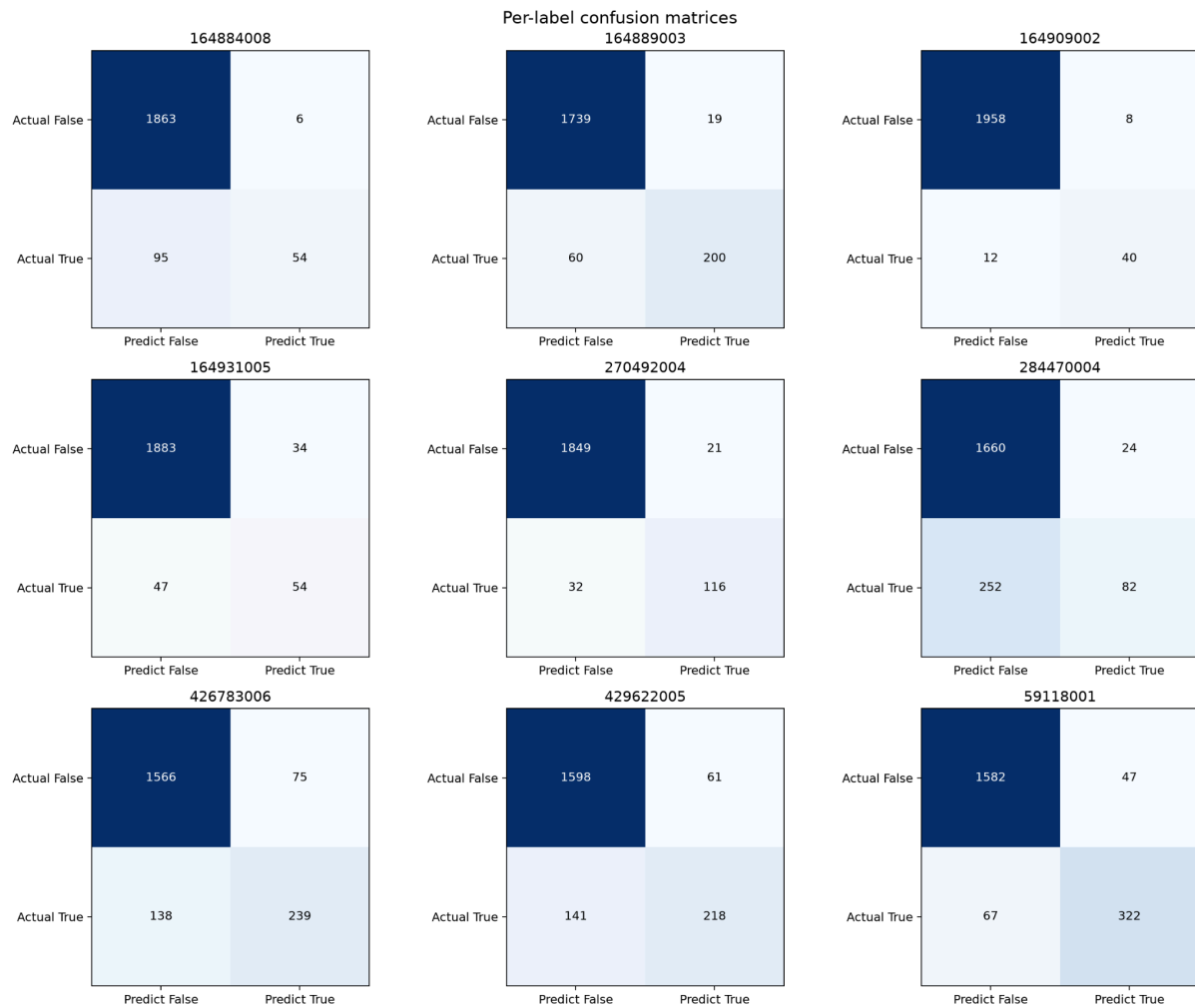


The application of this method had a small but noticeable positive impact on model performance (0.614 subset accuracy vs 0.584). This gain can likely be attributed to the oversampled cases having better representation that could be trained on

During this experiment it was noticed that our oversampling method had a flaw in it, using the incorrect label “270492004” instead of “284470004” as being a candidate for oversampling. After this mistake was recognised, a new dataset was constructed where 284470004 was properly oversampled and the experiment was repeated.

Experiment Name	cnn-oversample-l6-new
dataset	precomputed-split-gen-length-6
Model	2d-CNN
Variant	default
iterations	20



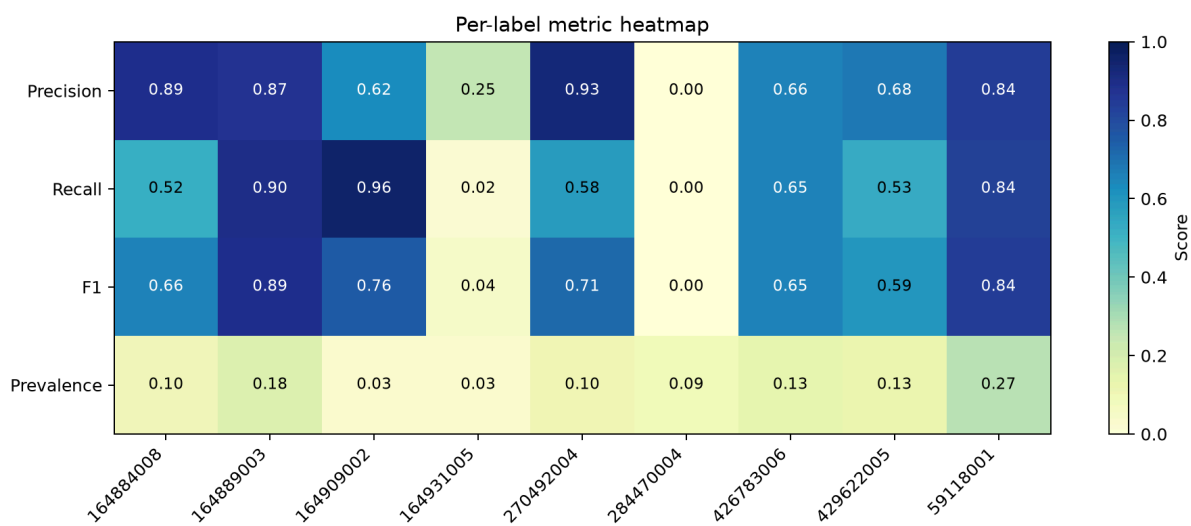
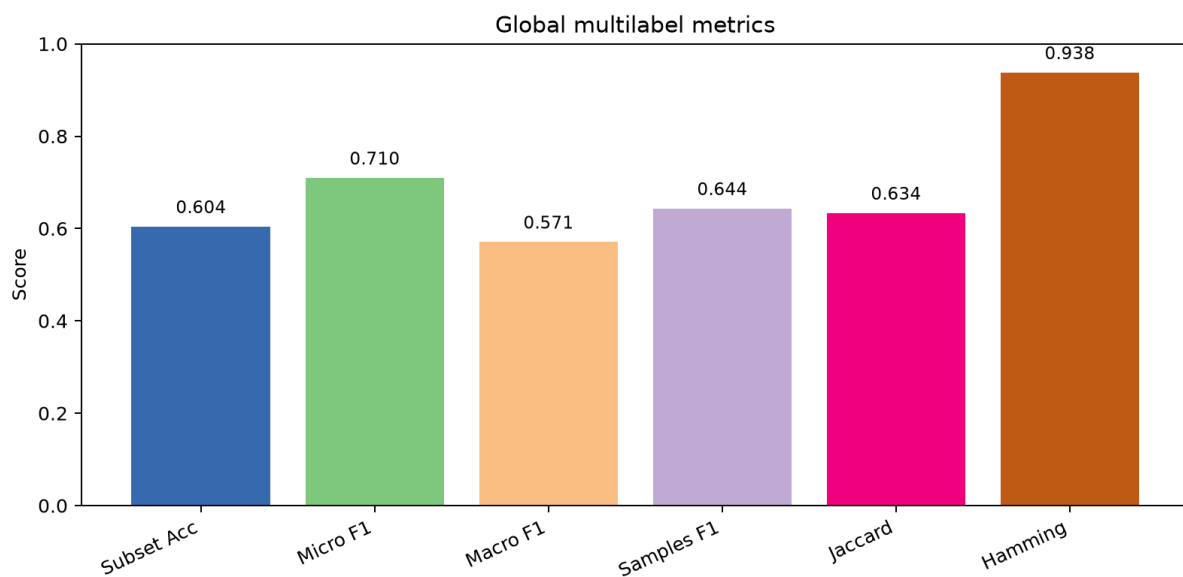


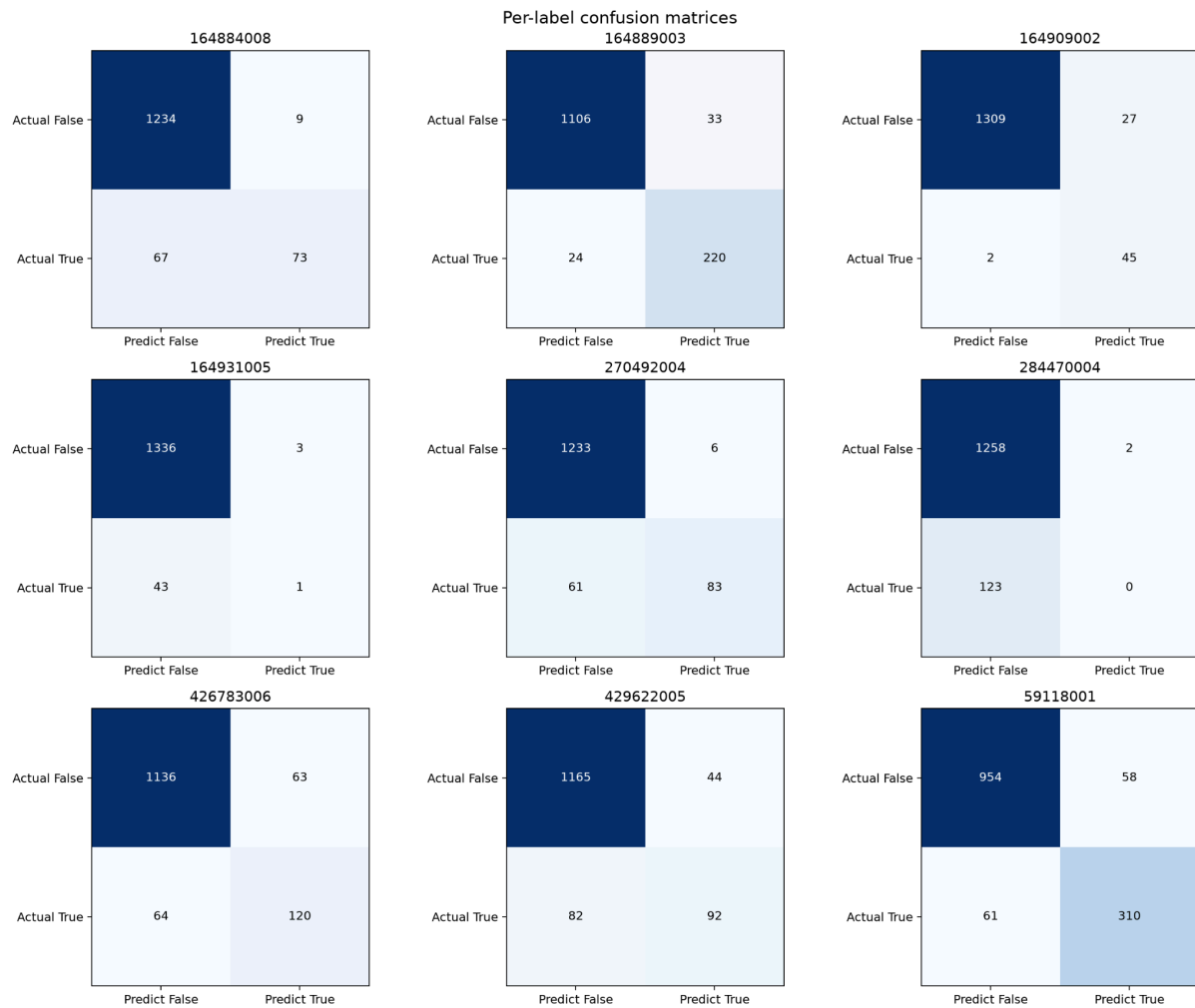
This variant where the oversampling was “properly” implemented saw worse performance than baseline (0.570 vs 0.584) with several individual classes falling behind, as a result we decided to continue with the original oversampling V1 algorithm. This is the reason behind there being two versions V1 and V2.

Longer Length

Another attempt to improve performance was to increase the length to allow (in theory) for more characteristics to be used for training.

Experiment Name	cnn-default-l10
dataset	precomputed-length-10
Model	2d-CNN
Variant	default
iterations	20



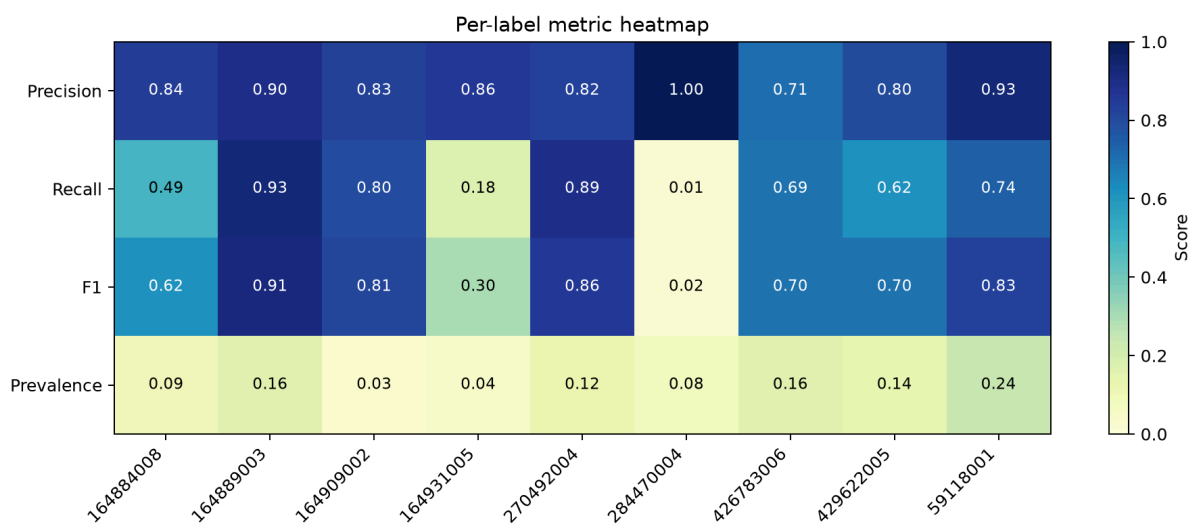
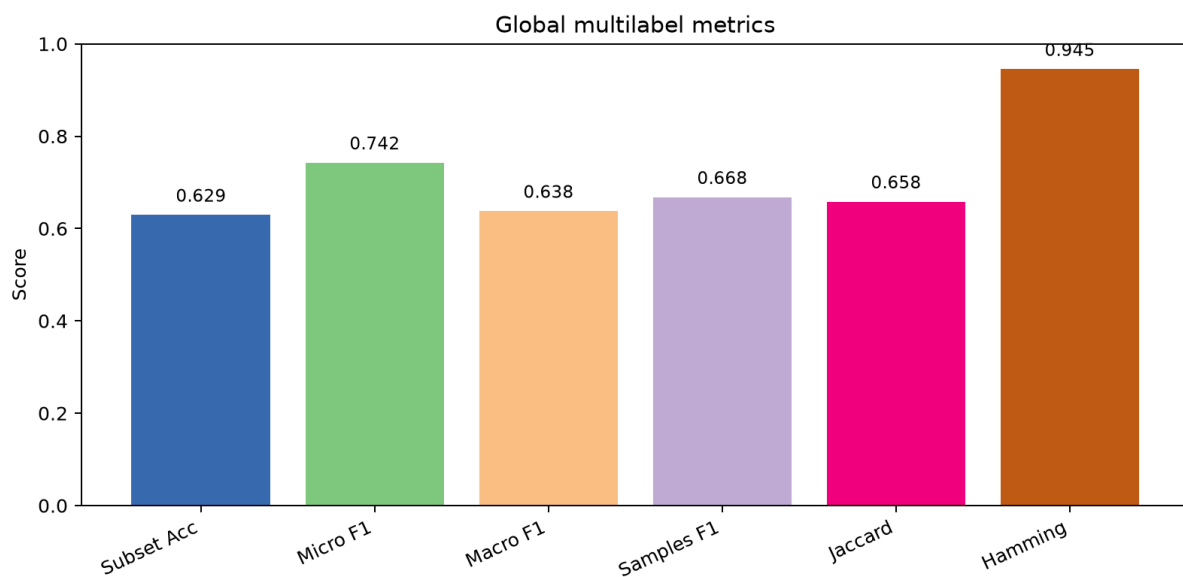


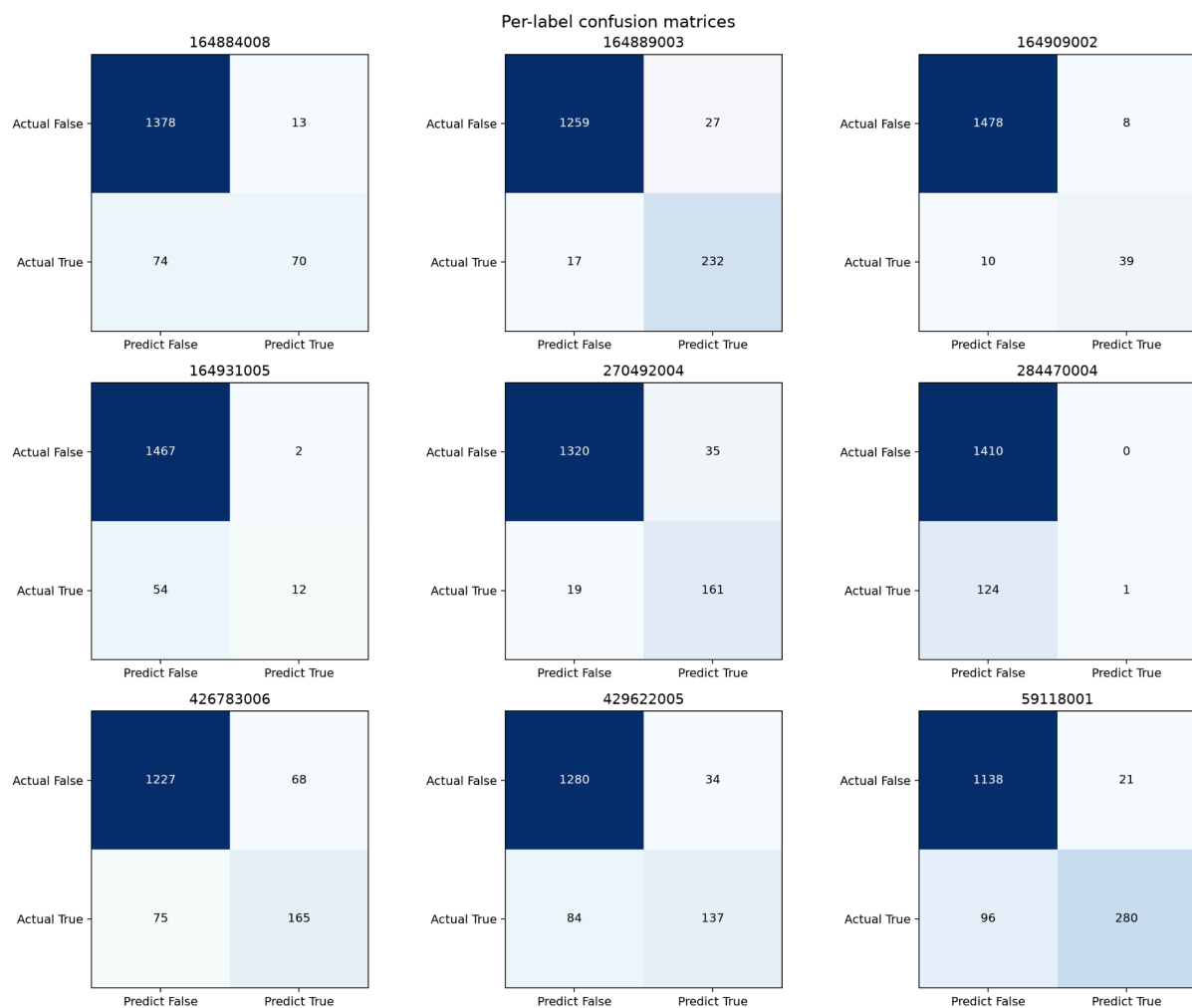
In practice the impact was negligible, but positive in nature (0.604 subset accuracy vs 0.584). However it should be noted that for the class involved in the mistake with oversampling, it was unable to recognize a single instance of it, resulting in a massive drop of Macro F1 score

Oversampling + Longer Length

As an experiment both strategies were combined to see whether the performance improvements would compound

Experiment Name	cnn-oversample-l10
dataset	precomputedsplit-genlength-10
Model	2d-CNN
Variant	default
iterations	20





The results were unsurprising with the overall metrics seeing a decent baseline improvement (0.629 subset accuracy vs 0.614 for oversampling and 0.604 for 10s length)

These improvements were compounding and the performance improvement was higher than the individual optimizations

LSTM

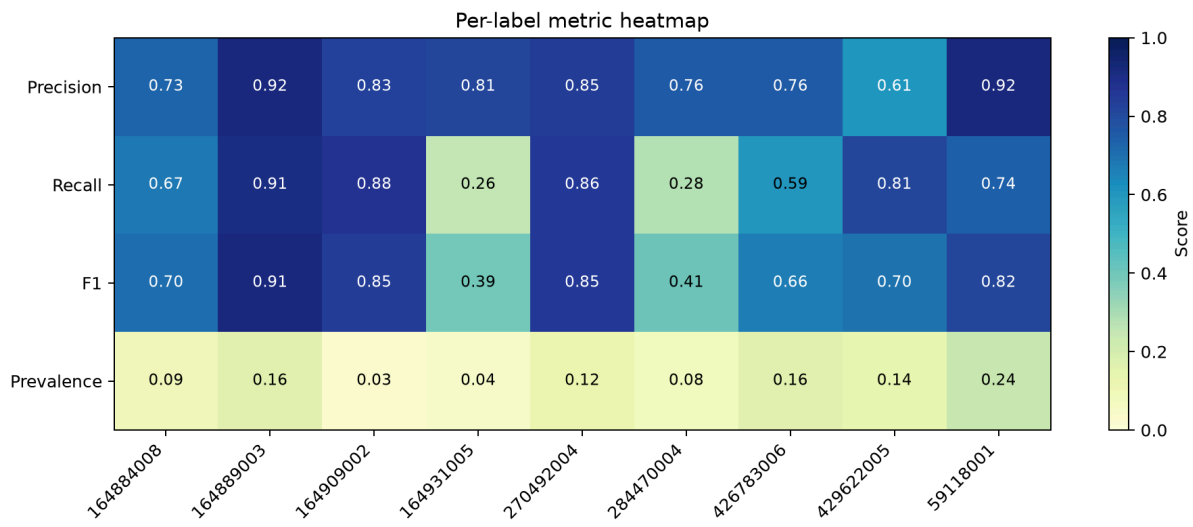
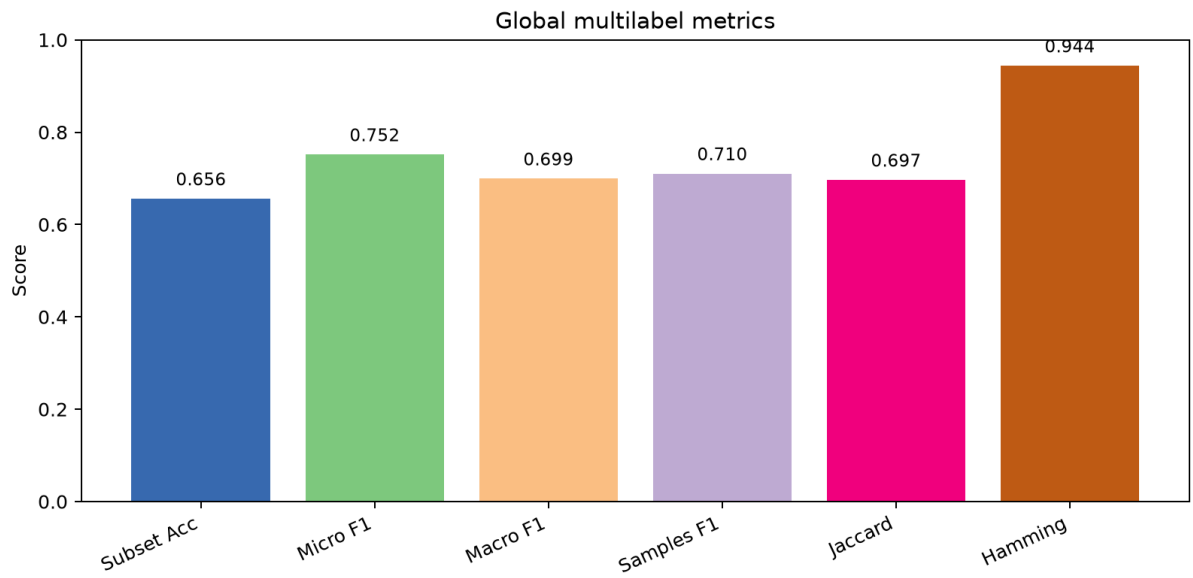
In order to refine our implementation we augmented our model layout by adding a singular LSTM layer before the final dense layer, this required augmenting remaining layers with the TimeDistributed layer, allowing them to work with 4d tensors that include the time dimension.

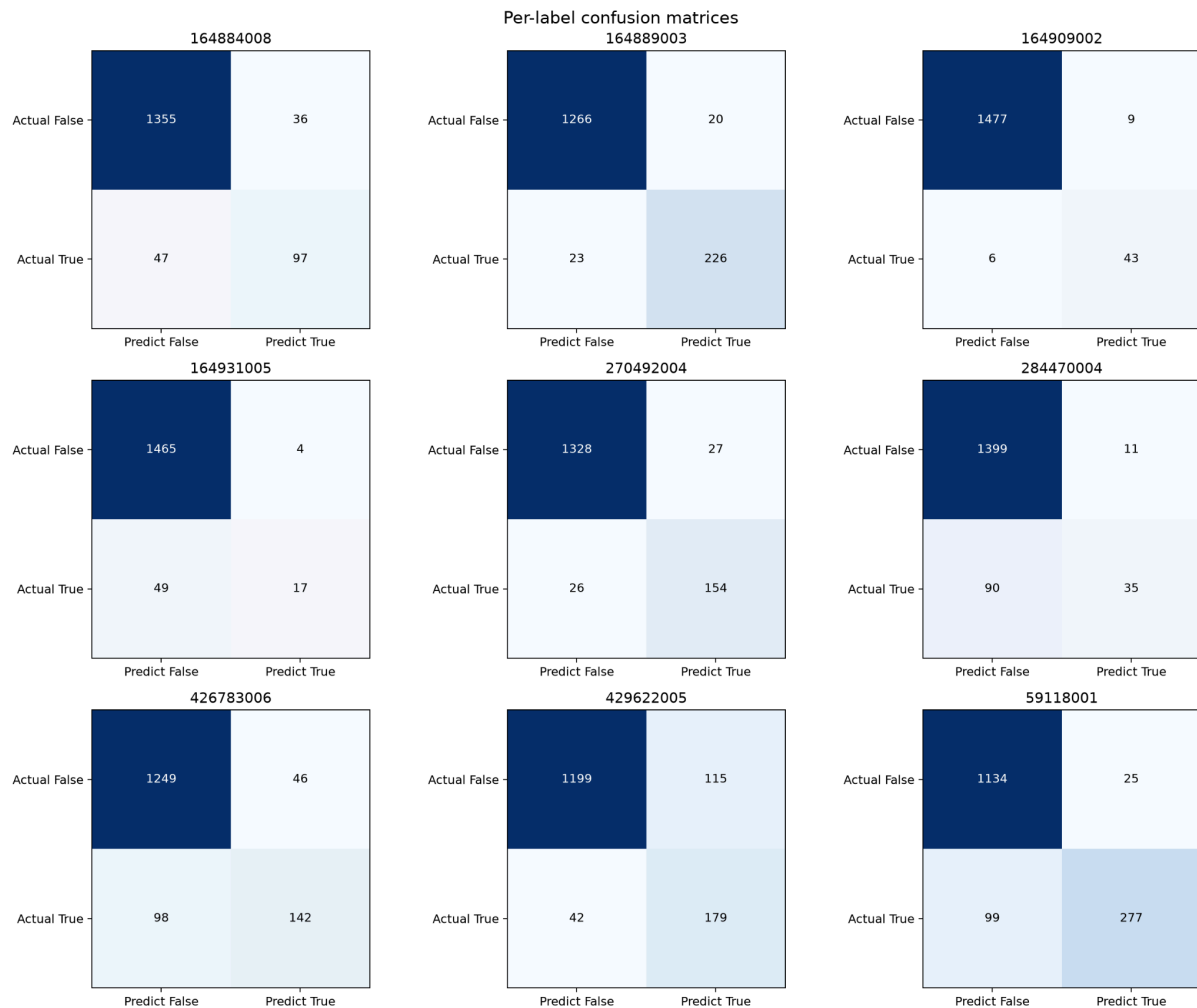
LSTM (Dimensional)

In the dimensional LTSM we take the unorthodox approach of using the leads themselves as the temporal dimension, in theory this could allow for our model to train off of the interdependency between all leads combined

Experiment Name	cnn-ltstm-l10-oversample
dataset	precomputedsplit-genlength-10
Model	2d-CNN (LSTM)

Variant	ltsm
iterations	18





This model exhibited the best performance across all of the experiments (0.656 vs 0.629 with cnn/oversample/l10), excelling in all metrics.

However the drawback was that it was extremely computationally expensive, warranting the usage of CUDA GPU acceleration.

This proved problematic as the newest versions of tensorflow do not support windows devices, and a complicated WSL2 setup using an older version of tensorflow had to be used instead to run model training.

It could be argued that the gains were not significant enough to justify the amount of computation and effort required to achieve them.

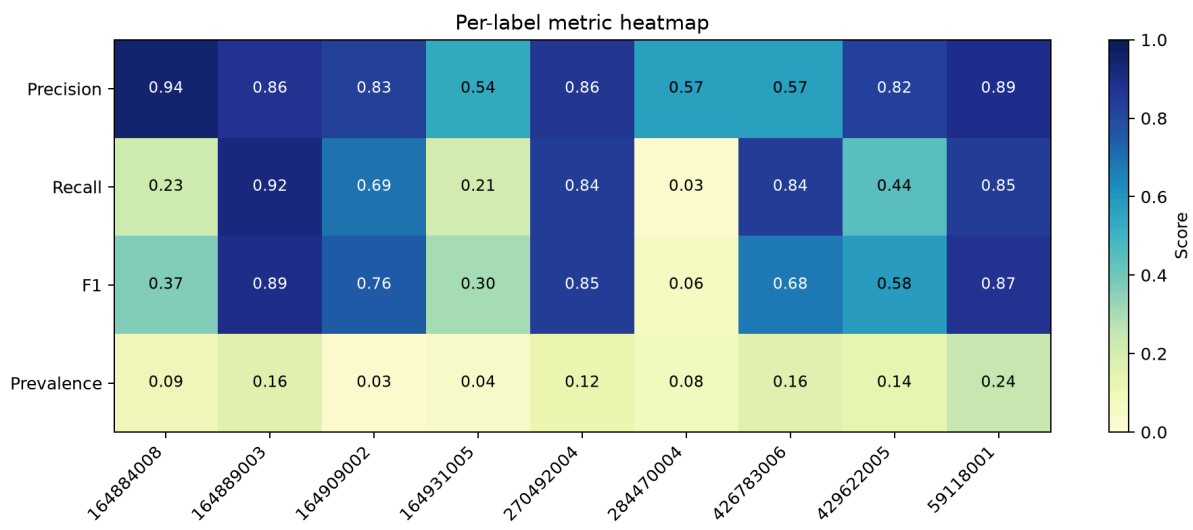
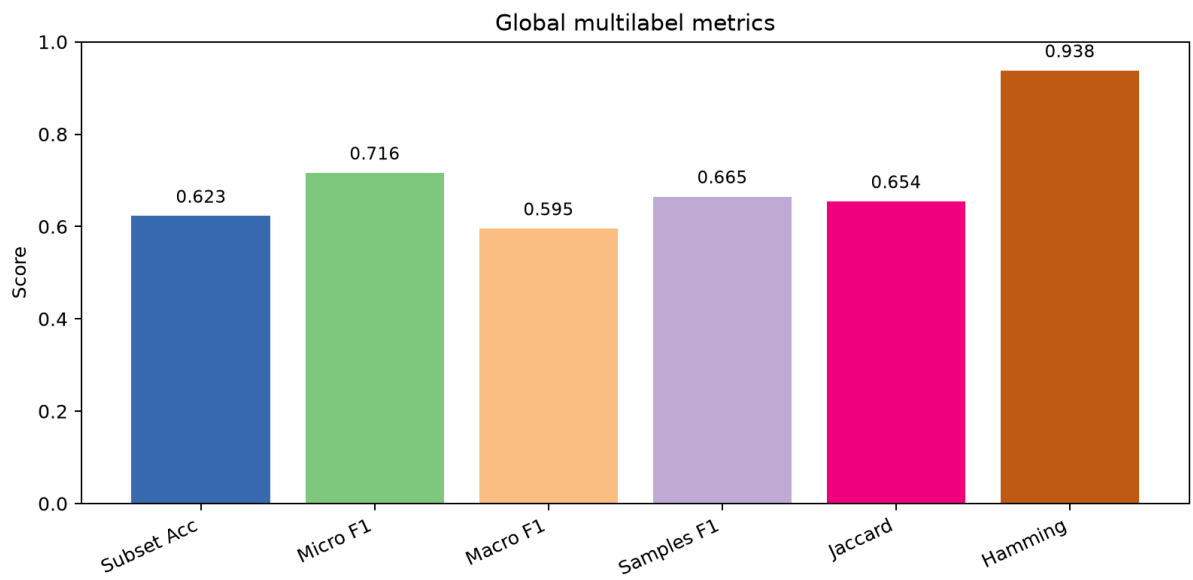
LSTM (Temporal)

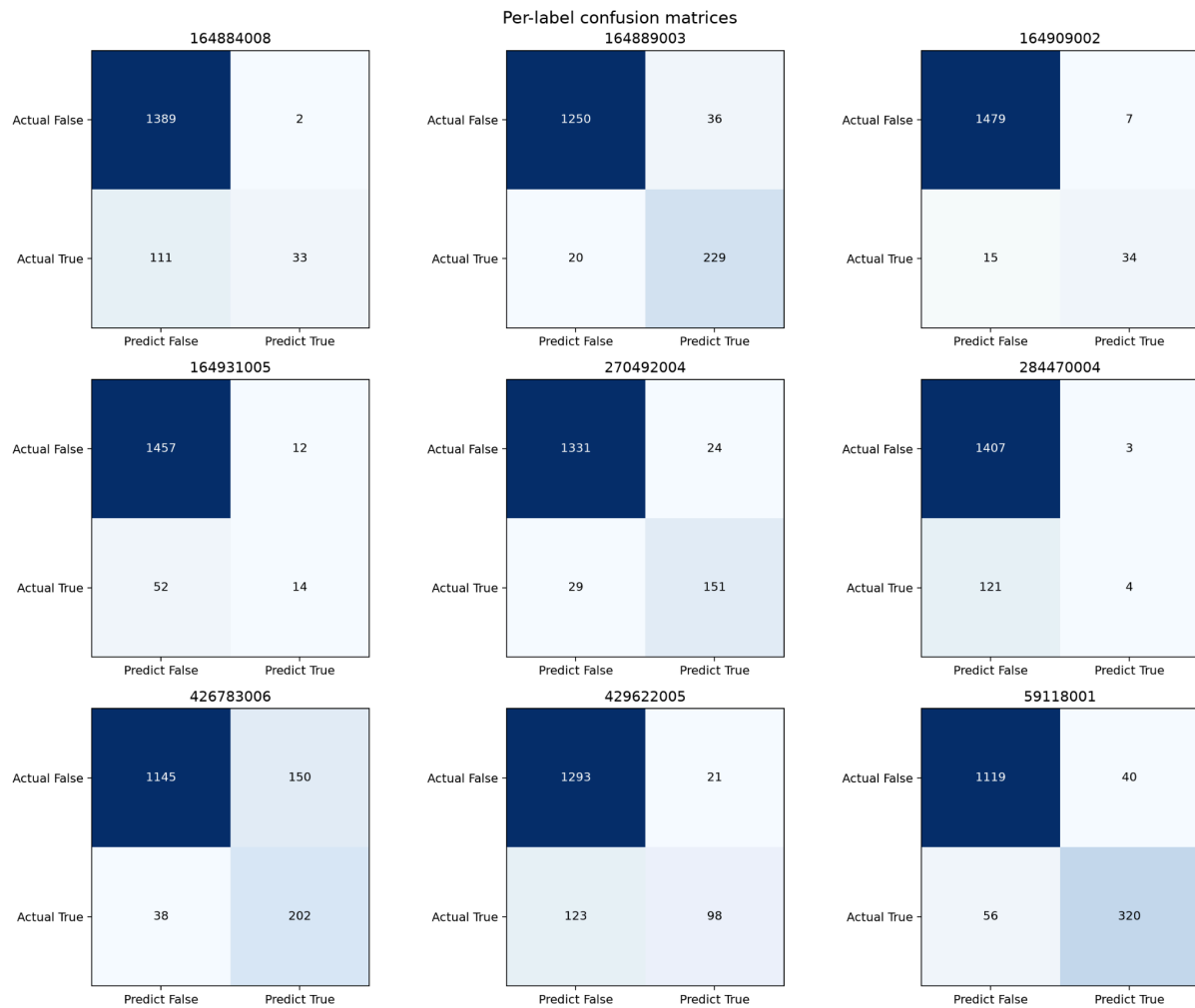
The original premise for LSTM was to use it for the intended purpose of applying it to the time dimension itself.

After the alternative architecture was explored we went back and implemented the intended LSTM architecture as well to compare.

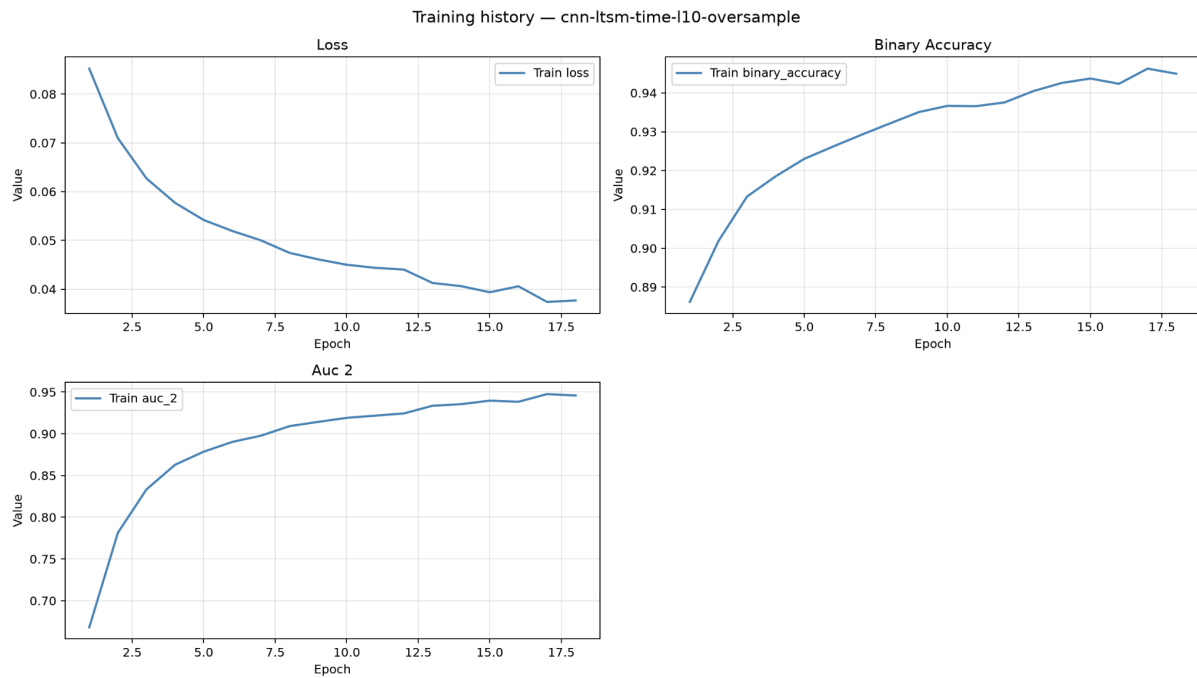
Experiment Name	cnn-ltsm-time-l10-oversample
-----------------	------------------------------

dataset	precomputedsplit-genlength-10
Model	2d-CNN (LSTM)
Variant	ltsm-time
iterations	18



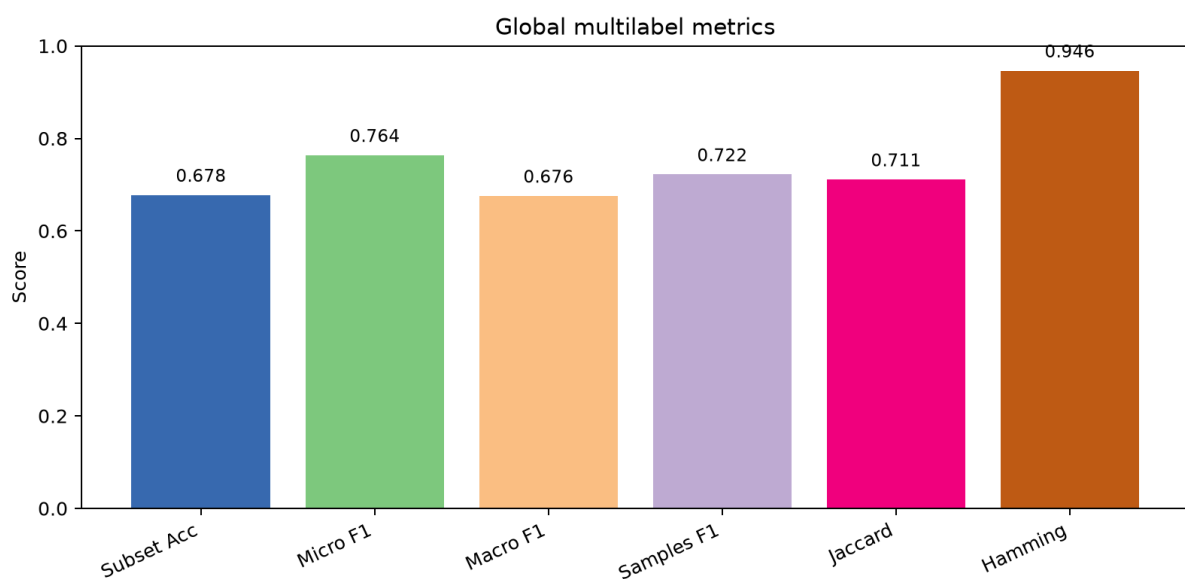


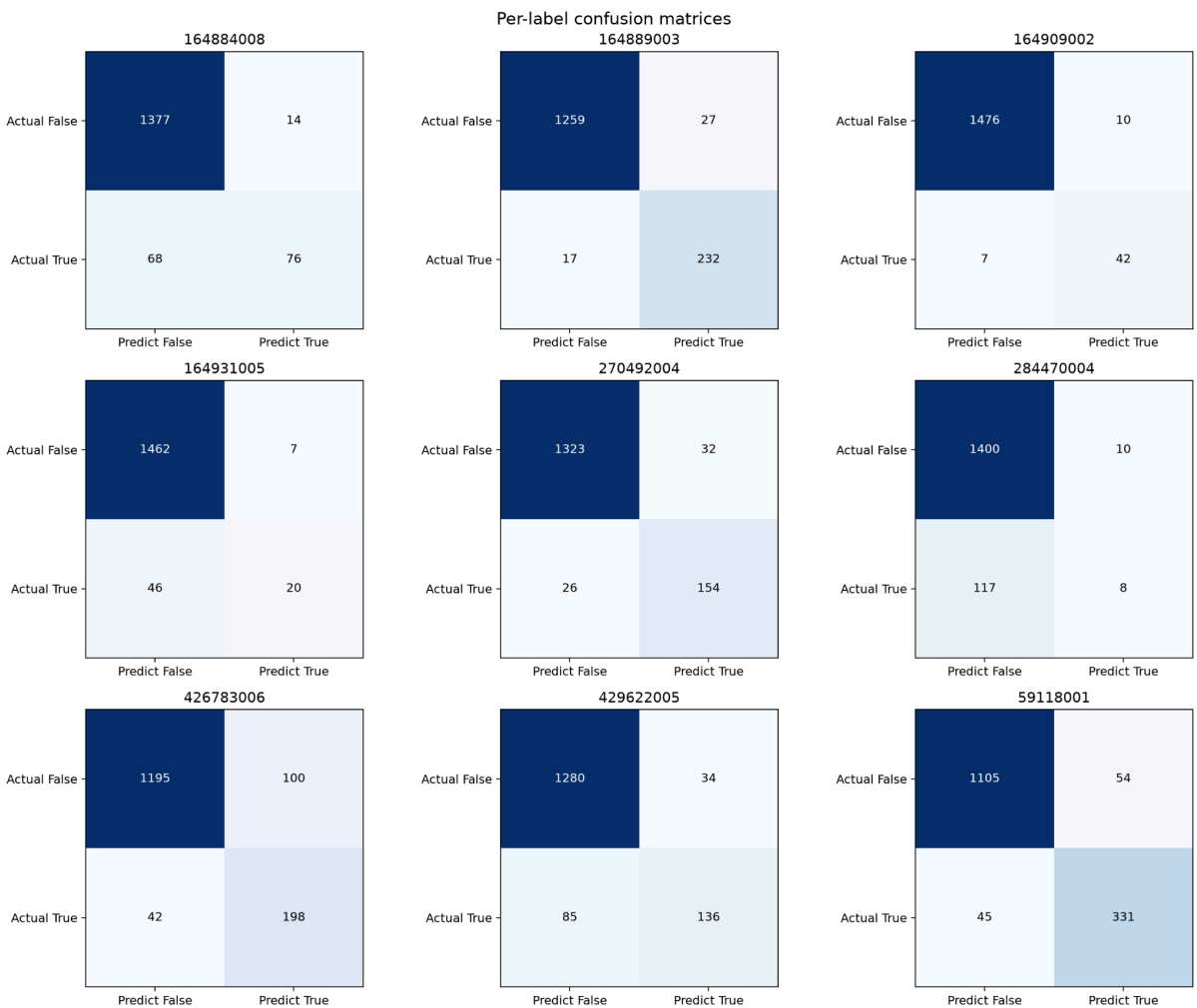
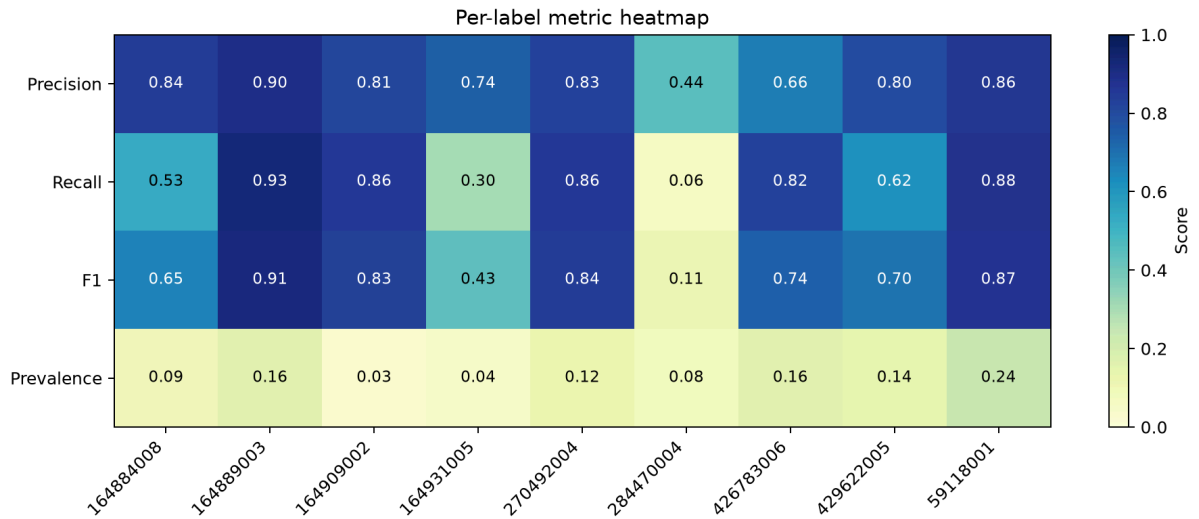
This experiment resulted in worse performance than the regular cnn and dimensional lstm, the performance being worse in comparison to the cnn was surprising as well suggesting a more fundamental flaw.



The training history suggests the training was not fully done yet and therefore it was repeated with a slightly higher maximum iteration count to rule that out as a factor

Experiment Name	cnn-lstm-time-l10-oversample-it25
dataset	precomputedsplit-genlength-10
Model	2d-CNN (LSTM)
Variant	lstm-time
iterations	25



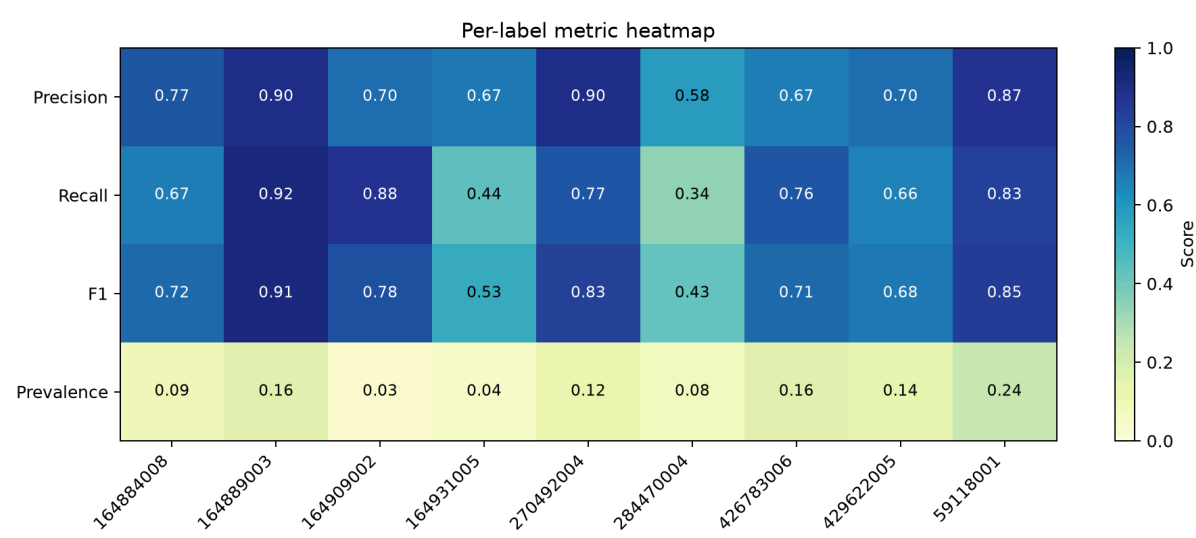
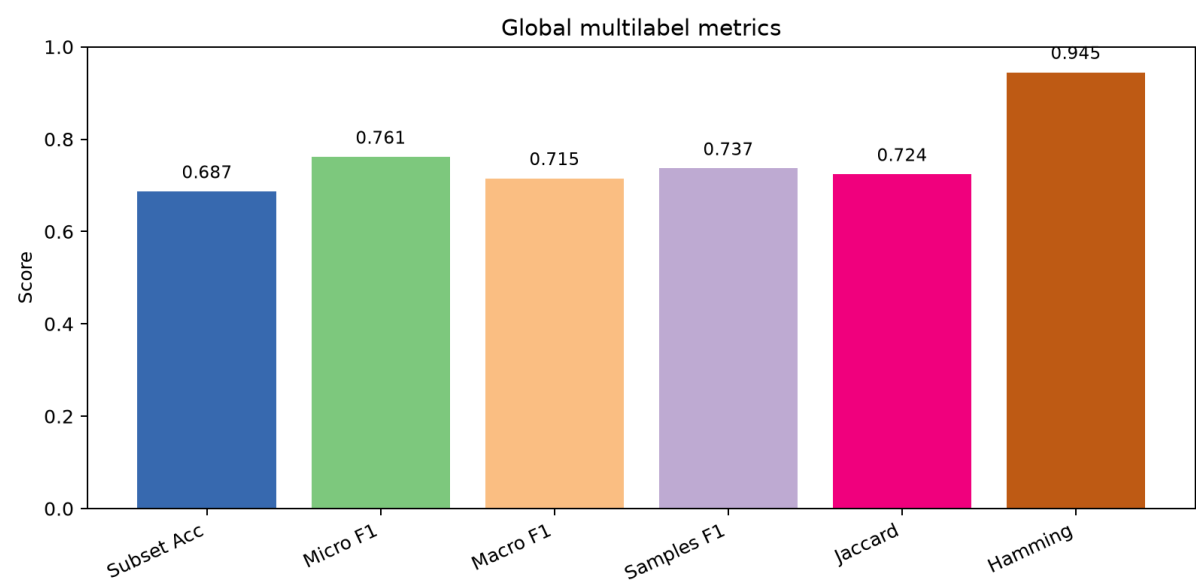


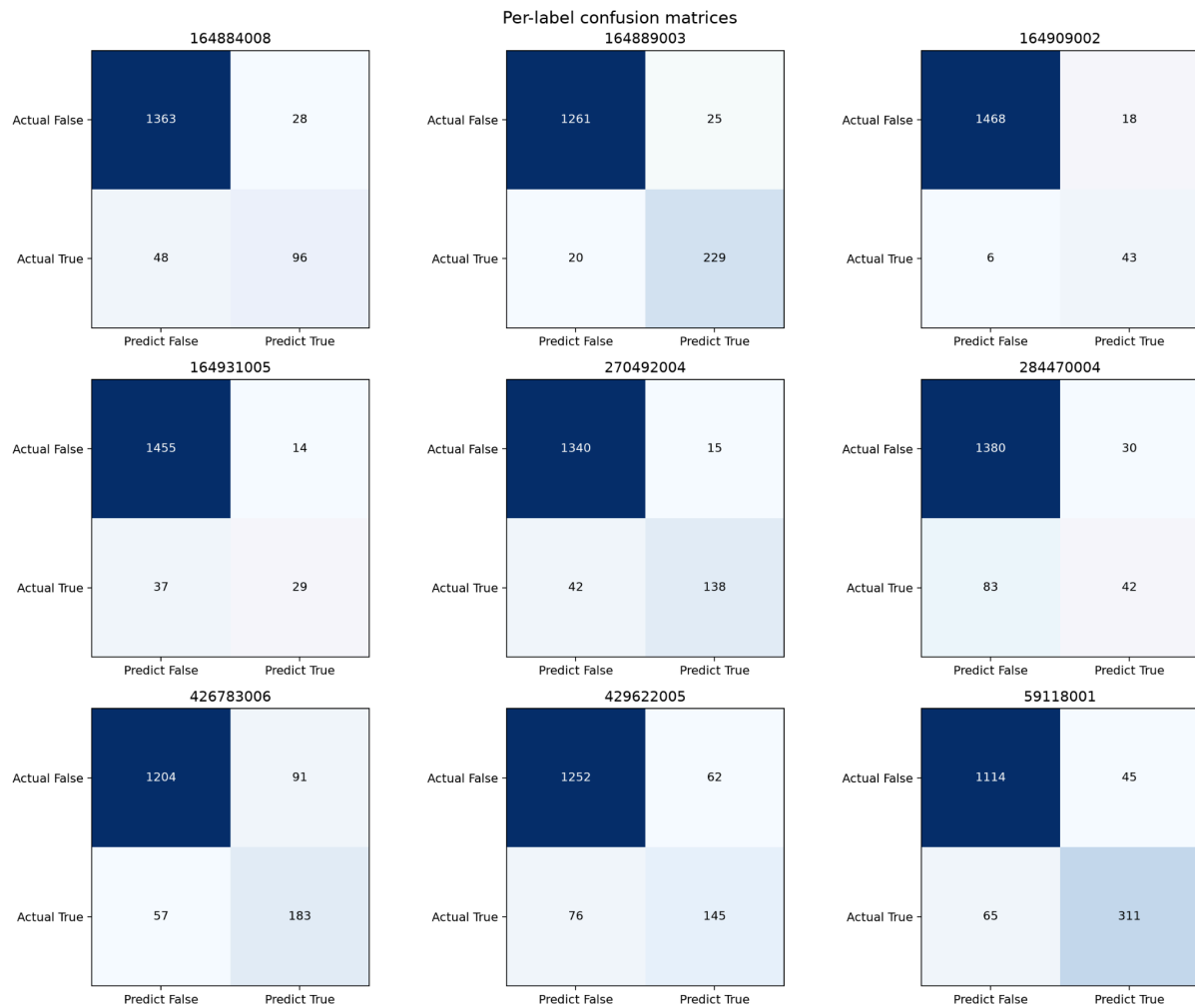
These results confirm the hypothesis, as after re-running the tests for an iteration count of 25, the model performance increased sharply and even outperformed the dimensional variant trained at 18.

It should be noted that increasing the epoch count to these numbers had negative effects on the regular variant of the 2d-cnn, and as a result the iteration variation was not further investigated for that model

However the mismatch in training epochs between the two LSTM models, warranted further investigation and that model was retrained for the increased epoch count to see how it fares

Experiment Name	cnn-ltsm-l10-oversample-it 25
dataset	precomputedsplit-genlength-10
Model	2d-CNN (LTSM)
Variant	ltsm-time
iterations	25





The dimensional LSTM variant still performed overall better than the temporal LSTM (0.687 vs 0.678) when the epoch count was increased, however the discrepancy between the loss in training and loss in tests was significant (0.0214 vs 0.0486) suggesting the model was struggling with overfitting.

Unfortunately the LSTM models were very computationally expensive to run 3 folds for and as a result we had to cap them at a certain epoch count.

It's likely the gains plateau at epoch count 50+, but we were unable to fully verify it.

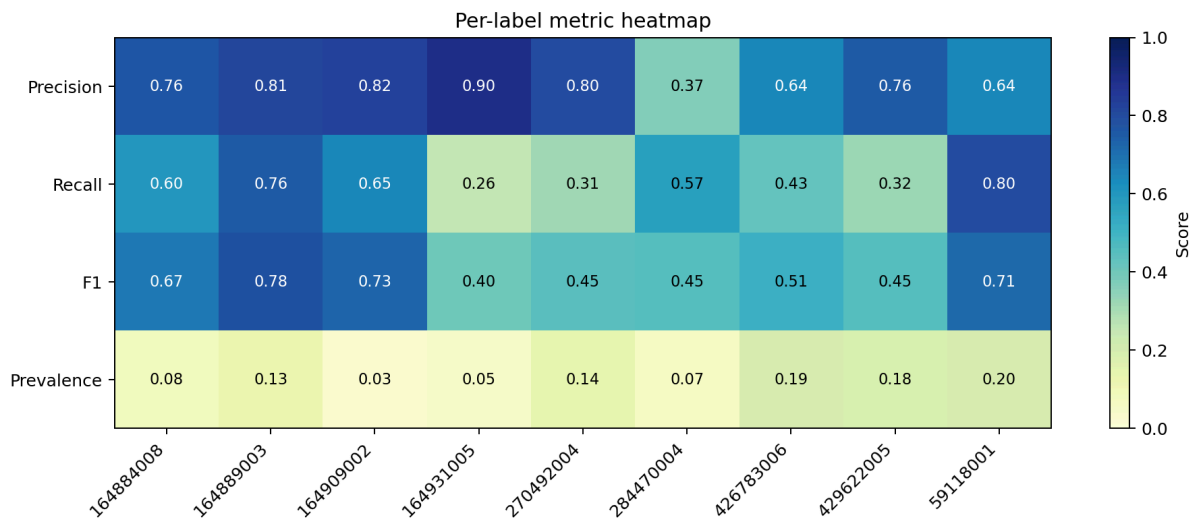
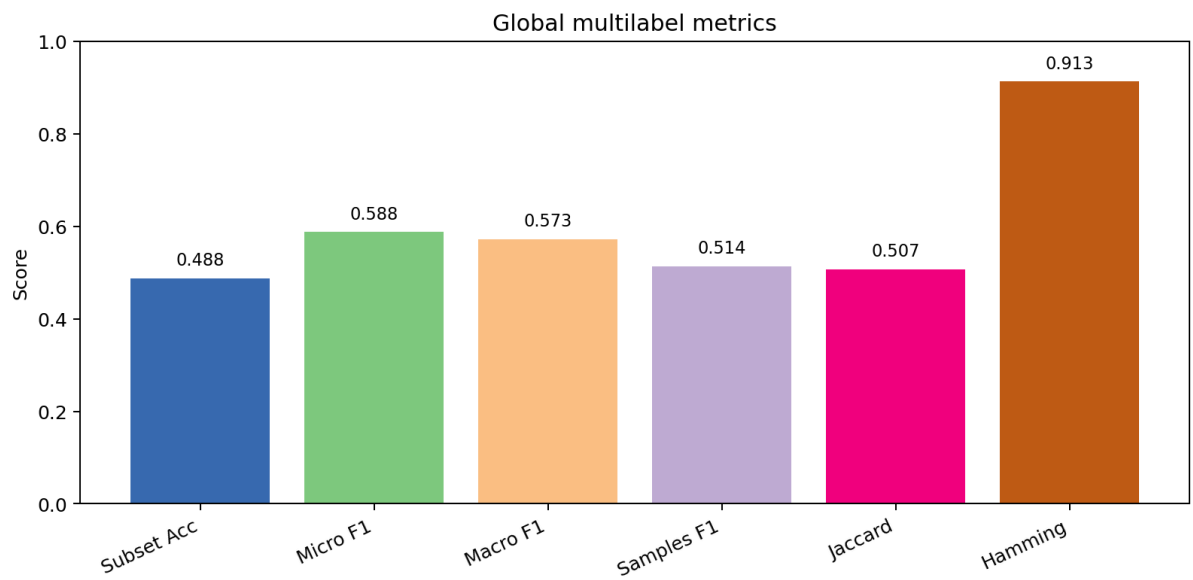
It can safely be assumed however that the temporal LSTM would eventually beat the dimensional variant due to overfitting issues.

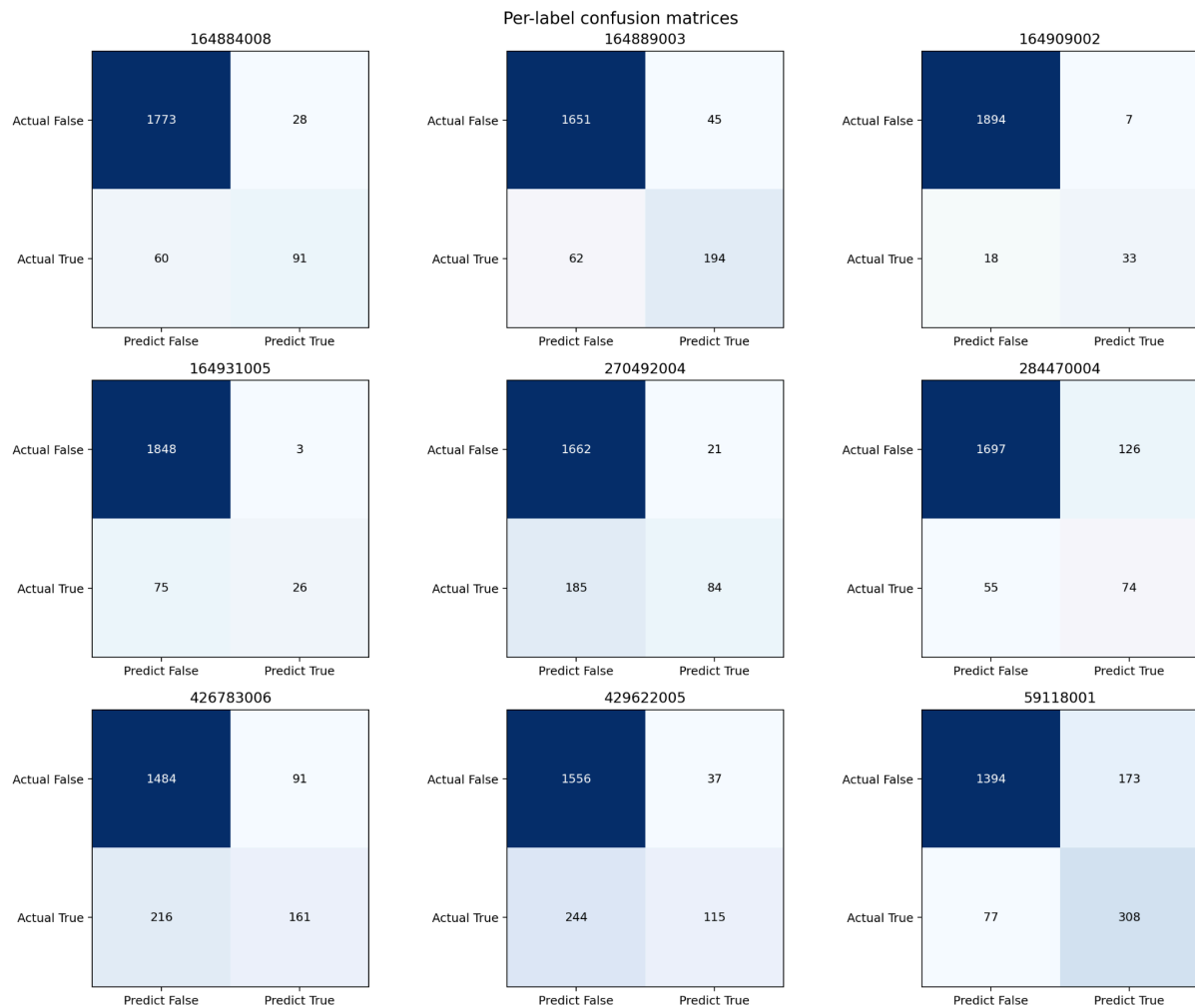
Other

While the custom 2D CNN was determined to be the best approach for this problem in the early stages of selection, we also trained the other two models on the complete dataset to assess their overall performance

SVM

Experiment Name	svm-default-l6-oversample
dataset	precomputed-split-gen-length-6-old
Model	SVM
Variant	default
iterations	auto - controlled by scikit



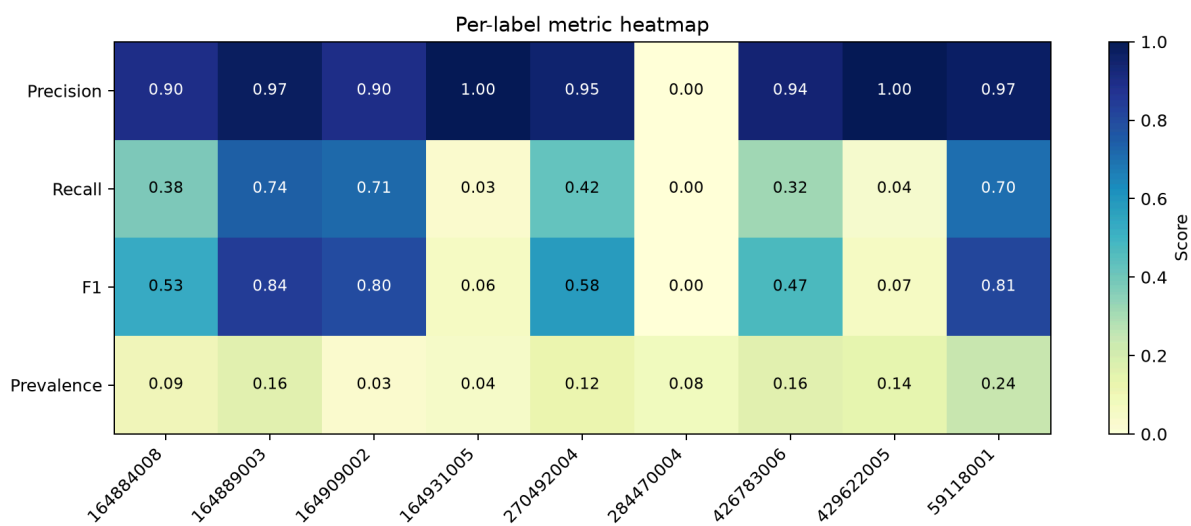
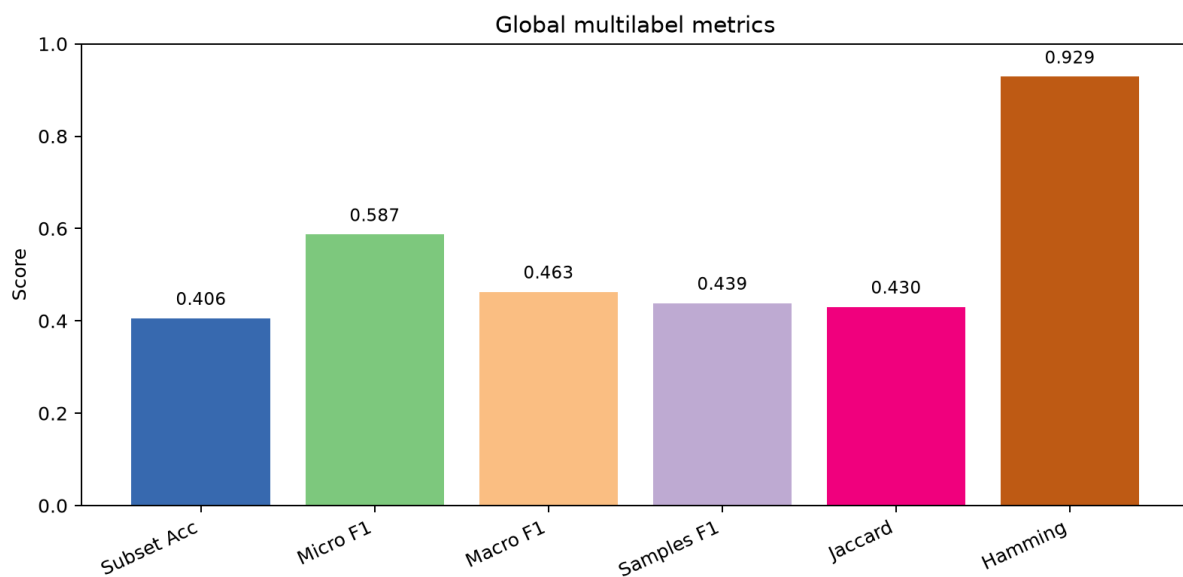


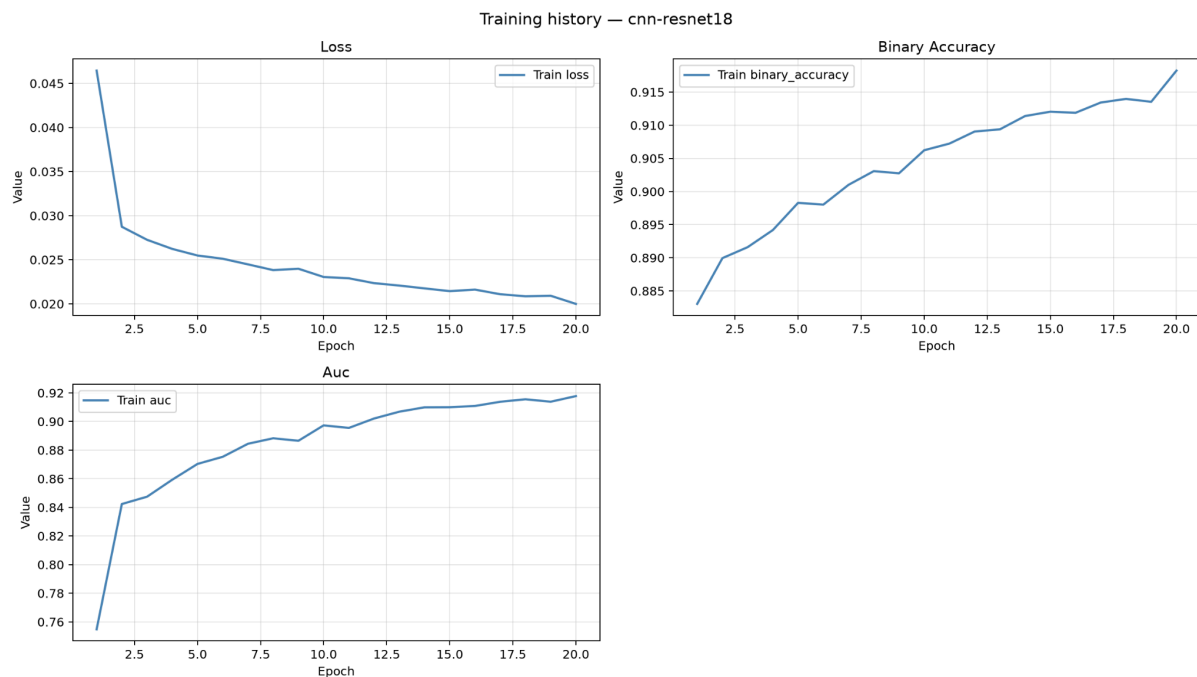
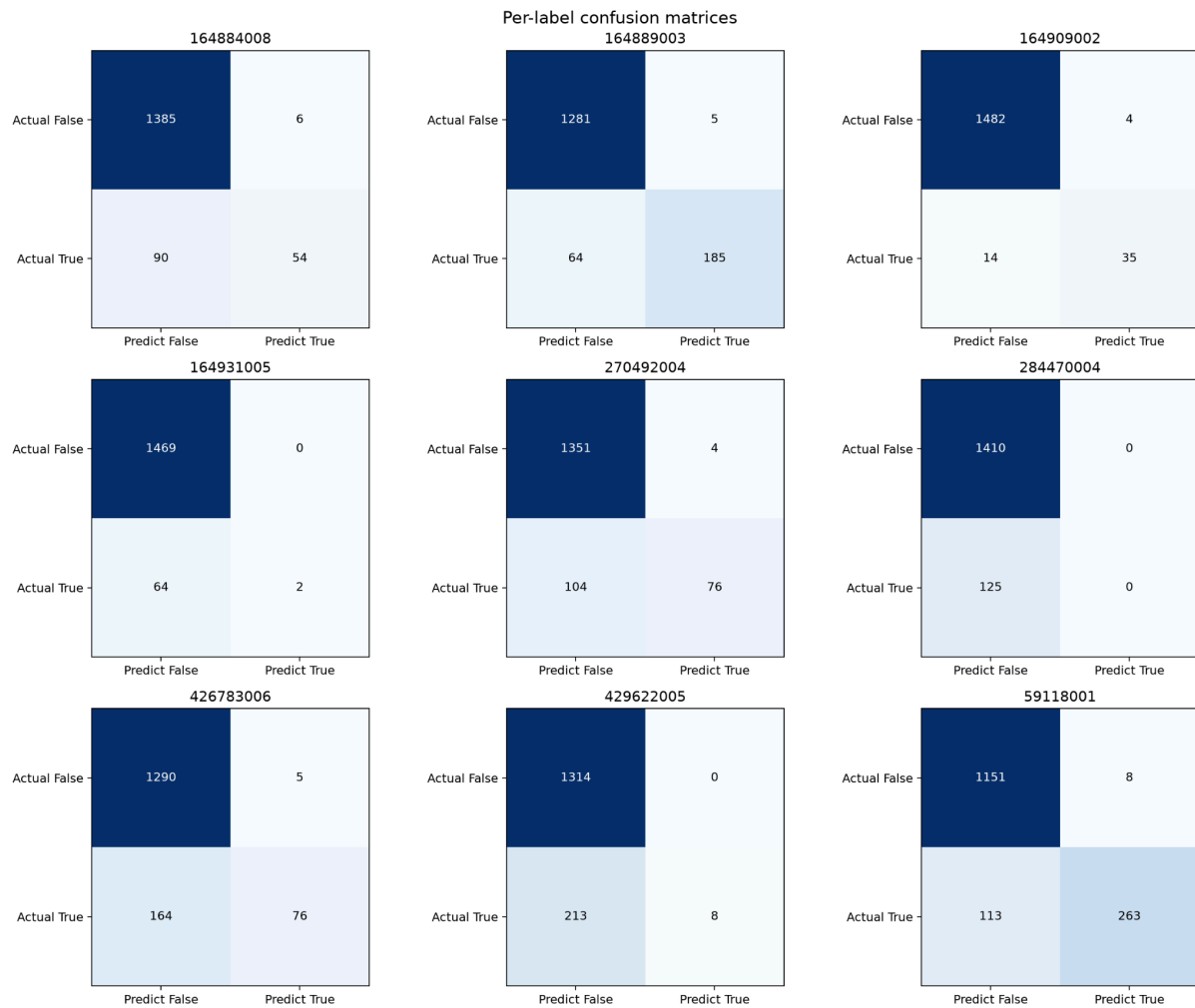
The SVM estimator powered multioutput classifier from scikit learn proved to be inadequate for the task, it generated several convergence warnings during training, and we were unable to use CUDA GPU acceleration for it, resulting in extremely slow training, despite using multithreading.

The actual performance was very poor, with the I10 variant having very low scores during training, as a result I6 was used for this example

ResNet 18

Experiment Name	resnet18-I10-oversample
dataset	precomputedsplit-genlength-10
Model	2d-CNN
Variant	resnet18
iterations	20





The resnet18 results were very peculiar, it had reached the lowest loss in training out of all the models, however in practice when using the constant thresholds as for all other

examples, it failed miserably in all possible categories, with excessive false predictions for classes where there were many true samples.

It is possible that implementing a threshold selection algorithm could possibly improve performance of this model.

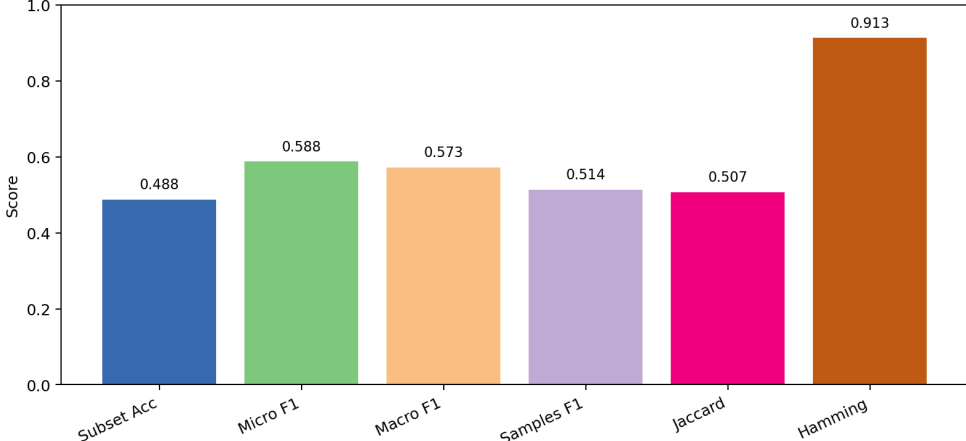
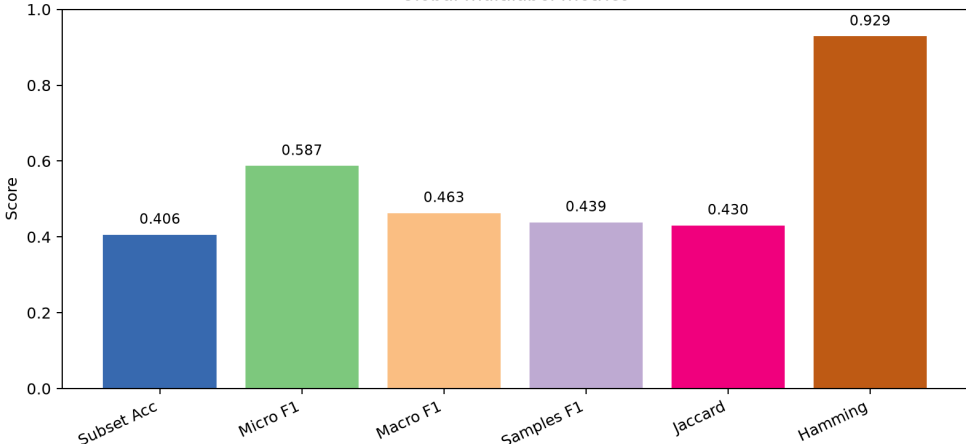
Conclusion

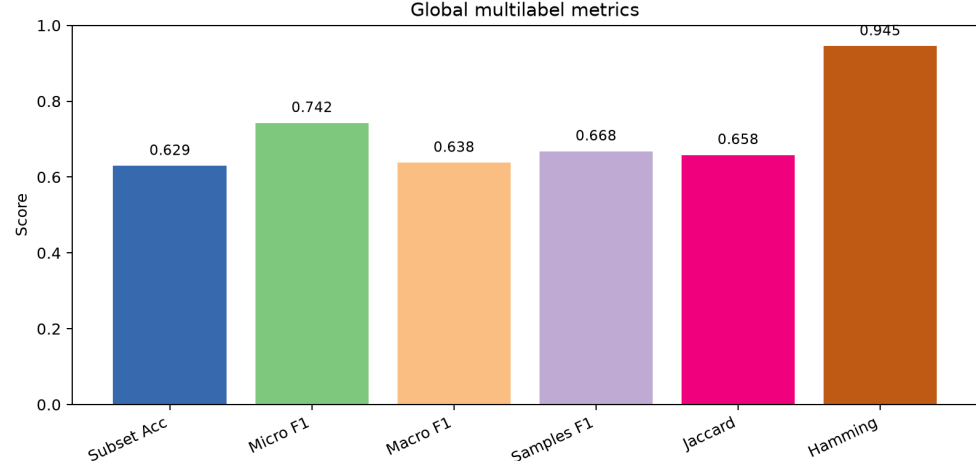
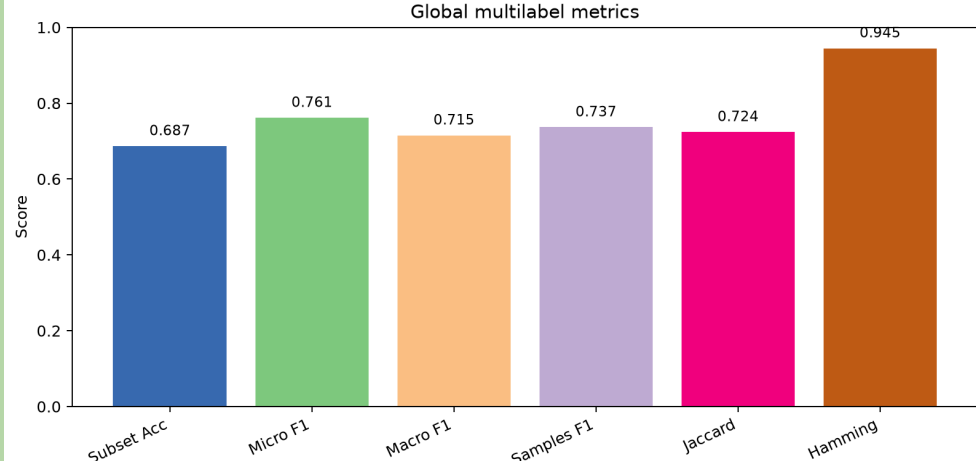
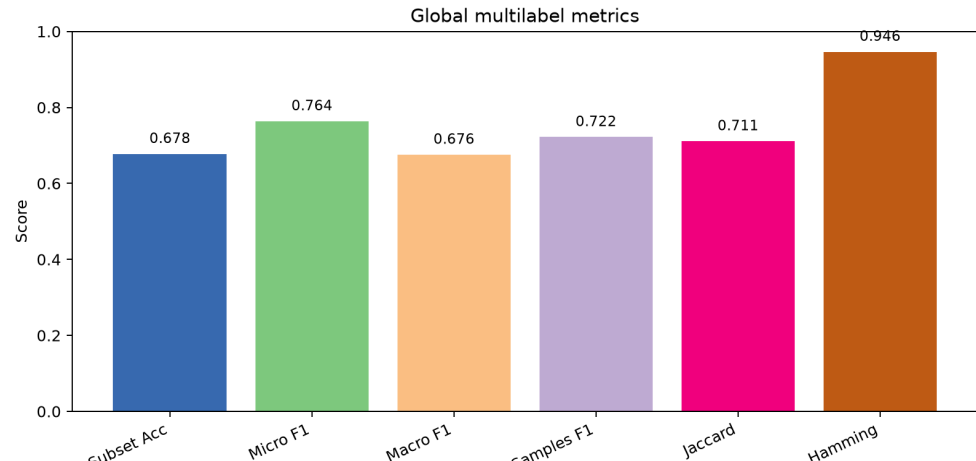
As part of the project we had to learn more advanced neural network concepts that were not originally covered during our course, however through trial and error and a healthy dose of experimentation we managed to arrive at a satisfactory result.

Our best performing model had an accuracy of 0.687 for all 9 labels and F1 scores within the 0.7 - 0.75 range which would place our solution in the top 20 of the original competition.

This is of course a bit deceptive of a metric as we removed a small subset of records which fell below 10s in length and we were not working with the exact same validation benchmark as the original competition, but regardless we are satisfied with this result.

An important aspect of this project was to research multiple neural networks and their utility for our classification task, leaving us with the following summary of the models and their respective performance in the best-case scenario:

Model type	Experiment	Results														
SVM	svm-default-l6-o versample	Global multilabel metrics <table><tr><th>Metric</th><th>Score</th></tr><tr><td>Subset Acc</td><td>0.488</td></tr><tr><td>Micro F1</td><td>0.588</td></tr><tr><td>Macro F1</td><td>0.573</td></tr><tr><td>Samples F1</td><td>0.514</td></tr><tr><td>Jaccard</td><td>0.507</td></tr><tr><td>Hamming</td><td>0.913</td></tr></table>	Metric	Score	Subset Acc	0.488	Micro F1	0.588	Macro F1	0.573	Samples F1	0.514	Jaccard	0.507	Hamming	0.913
Metric	Score															
Subset Acc	0.488															
Micro F1	0.588															
Macro F1	0.573															
Samples F1	0.514															
Jaccard	0.507															
Hamming	0.913															
ResNet18	resnet18-l10-ove rsample	Global multilabel metrics <table><tr><th>Metric</th><th>Score</th></tr><tr><td>Subset Acc</td><td>0.406</td></tr><tr><td>Micro F1</td><td>0.587</td></tr><tr><td>Macro F1</td><td>0.463</td></tr><tr><td>Samples F1</td><td>0.439</td></tr><tr><td>Jaccard</td><td>0.430</td></tr><tr><td>Hamming</td><td>0.929</td></tr></table>	Metric	Score	Subset Acc	0.406	Micro F1	0.587	Macro F1	0.463	Samples F1	0.439	Jaccard	0.430	Hamming	0.929
Metric	Score															
Subset Acc	0.406															
Micro F1	0.587															
Macro F1	0.463															
Samples F1	0.439															
Jaccard	0.430															
Hamming	0.929															

2D-CNN	cnn-oversample-l10	Global multilabel metrics  <table><tr><th>Metric</th><th>Score</th></tr><tr><td>Subset Acc</td><td>0.629</td></tr><tr><td>Micro F1</td><td>0.742</td></tr><tr><td>Macro F1</td><td>0.638</td></tr><tr><td>Samples F1</td><td>0.668</td></tr><tr><td>Jaccard</td><td>0.658</td></tr><tr><td>Hamming</td><td>0.945</td></tr></table>	Metric	Score	Subset Acc	0.629	Micro F1	0.742	Macro F1	0.638	Samples F1	0.668	Jaccard	0.658	Hamming	0.945
Metric	Score															
Subset Acc	0.629															
Micro F1	0.742															
Macro F1	0.638															
Samples F1	0.668															
Jaccard	0.658															
Hamming	0.945															
2D-CNN (LSTM - Dimensional)	cnn-ltsm-l10-oversample-it25	Global multilabel metrics  <table><tr><th>Metric</th><th>Score</th></tr><tr><td>Subset Acc</td><td>0.687</td></tr><tr><td>Micro F1</td><td>0.761</td></tr><tr><td>Macro F1</td><td>0.715</td></tr><tr><td>Samples F1</td><td>0.737</td></tr><tr><td>Jaccard</td><td>0.724</td></tr><tr><td>Hamming</td><td>0.945</td></tr></table>	Metric	Score	Subset Acc	0.687	Micro F1	0.761	Macro F1	0.715	Samples F1	0.737	Jaccard	0.724	Hamming	0.945
Metric	Score															
Subset Acc	0.687															
Micro F1	0.761															
Macro F1	0.715															
Samples F1	0.737															
Jaccard	0.724															
Hamming	0.945															
2D-CNN (LSTM - Temporal)	cnn-ltsm-time-l10-oversample-it25	Global multilabel metrics  <table><tr><th>Metric</th><th>Score</th></tr><tr><td>Subset Acc</td><td>0.678</td></tr><tr><td>Micro F1</td><td>0.764</td></tr><tr><td>Macro F1</td><td>0.676</td></tr><tr><td>Samples F1</td><td>0.722</td></tr><tr><td>Jaccard</td><td>0.711</td></tr><tr><td>Hamming</td><td>0.946</td></tr></table>	Metric	Score	Subset Acc	0.678	Micro F1	0.764	Macro F1	0.676	Samples F1	0.722	Jaccard	0.711	Hamming	0.946
Metric	Score															
Subset Acc	0.678															
Micro F1	0.764															
Macro F1	0.676															
Samples F1	0.722															
Jaccard	0.711															
Hamming	0.946															

During the course of this project we had significant technical difficulties which we had to troubleshoot, but through additional research on our part we were able to overcome and resolve them.

Between the strategies we researched for improving our scores, alternative model types, iterating upon our ideas, and most importantly problem solving and adaptation after failures we got a pretty solid crash course on more advanced machine learning topics.